



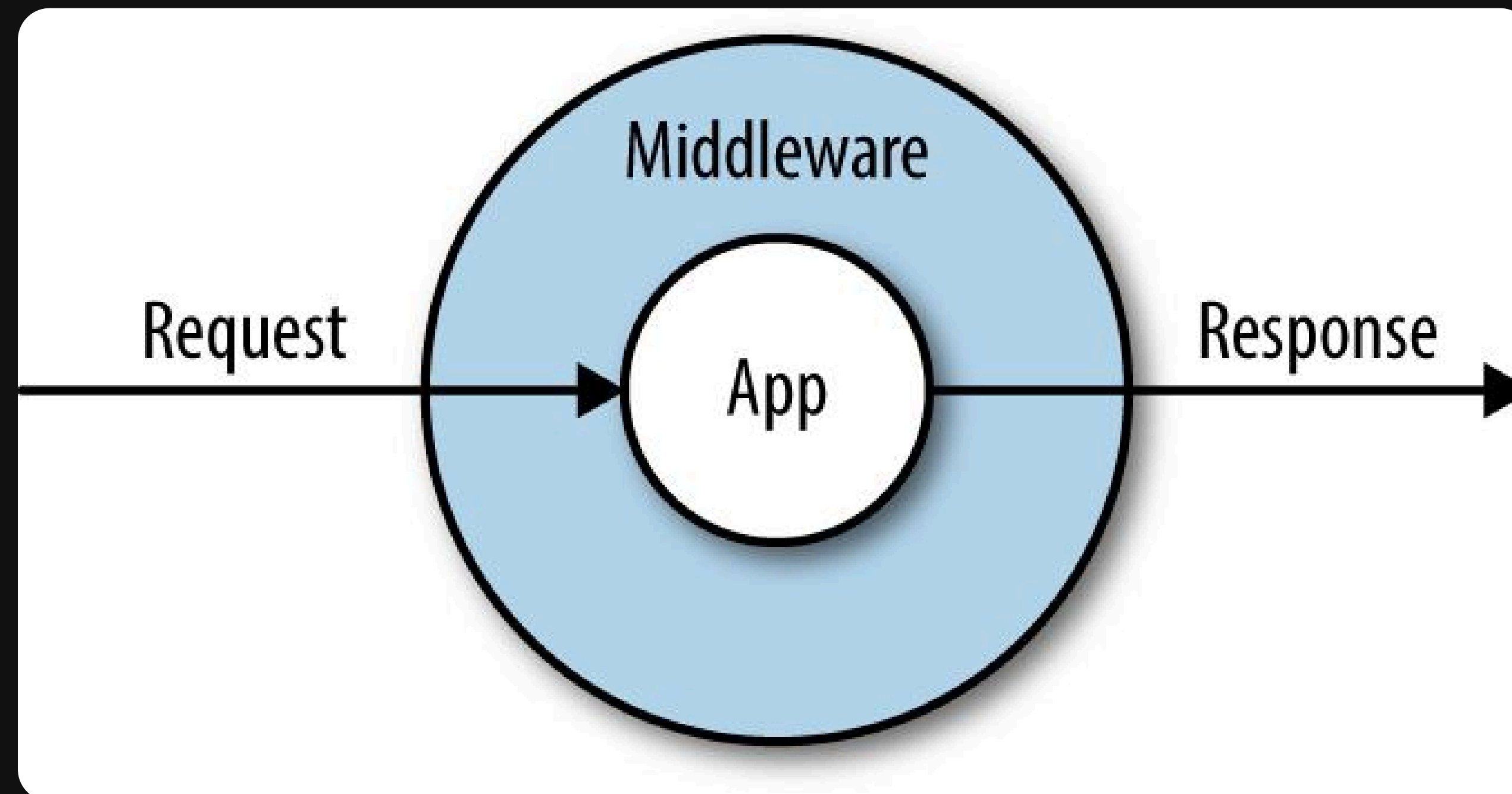
Server-Side Web Development (06016418)



วันนี้เราจะได้เรียนอะไรกันบ้าง ?



Middleware





Middleware

Middleware คือฟังก์ชันที่เป็นเสมือน "ด่านตรวจ" หรือตัวกลางในการจัดการคำขอ (Request) จากฝั่งผู้ใช้งาน (Client) ก่อนที่จะส่งต่อไปยังปลายทางในเซิร์ฟเวอร์ของเรา

หน้าที่หลักของ Middleware :

- การเข้าถึงข้อมูล : Middleware สามารถอ่าน แก้ไข หรือตรวจสอบข้อมูลภายใน request object และ response object ได้
- การควบคุมการทำงาน : หากเราต้องการเพิ่มฟีเจอร์อย่างการตรวจสอบสิทธิ์ (Authentication), การบันทึกข้อมูลเข้าสู่ระบบ (Logging), หรือการบีบอัดข้อมูล เราสามารถทำผ่าน Middleware ได้โดยไม่ต้องเขียนโค้ดซ้ำซ้อนในทุกๆ Route
- การส่งต่อ (Next) : Middleware แต่ละตัวจะรับค่า next ฟังก์ชันมาด้วย ซึ่งสำคัญมาก เพราะหากเราไม่เรียกใช้ next() กระบวนการทำงานจะถูกหยุดค้างไว้และไม่ส่งต่อไปยังจุดถัดไป



```
const express = require('express');
const app = express();

// 1. สร้าง Logger Middleware Function
const simpleLogger = (req, res, next) => {
  const method = req.method;           // GET, POST, ฯลฯ
  const url = req.url;                 // Path เช่น /about
  const timestamp = new Date().toISOString(); // เวลาปัจจุบัน

  // แสดงผลใน Console
  console.log(`[${timestamp}] ${method} request to: ${url}`);

  // สำคัญมาก! ต้องเรียก next() เพื่อส่ง Request ไปยัง Middleware/Route ถัดไป
  next();
};

// 2. เรียกใช้งาน Middleware สำหรับทุก Request (Global Middleware)
app.use(simpleLogger);

// 3. กำหนด Route สำหรับทดสอบ
app.get('/', (req, res) => {
  res.send('หน้าหลัก (Home Page)');
});

app.get('/users', (req, res) => {
  res.send('หน้าข้อมูลผู้ใช้ (Users Page)');
});

// 4. เริ่มต้น Server
app.listen(3000, () => {
  console.log('Server is running on http://localhost:3000');
});
```

<https://goonlinetools.com/snapshot/code/#ac7qqj3ol1sq4edblpagm>



Environment Variables

A large yellow square with rounded corners, containing the text `.ENV` in bold black font, representing an environment variable file.

.ENV



Environment Variables

Environment Variables เป็นตัวแปรที่ถูกกำหนดไว้ในระบบปฏิบัติการหรือเซิร์ฟเวอร์ที่รันแอปพลิเคชันของเรา โดยมักใช้สำหรับเก็บข้อมูลที่เป็นความลับ เช่น รหัสผ่านฐานข้อมูล, คีย์ API, หรือการตั้งค่าอื่น ๆ ที่เราไม่ต้องการให้เผยแพร่แก่ผู้ใช้งานหรือผู้ไม่หวังดี

ประโยชน์ของ Environment Variables

- ความปลอดภัย : Environment Variables ช่วยลดความเสี่ยงในการเปิดเผยข้อมูลที่เป็นความลับ เนื่องจากข้อมูลเหล่านี้จะไม่ถูกเก็บในโค้ดของแอปพลิเคชันโดยตรง
- ความยืดหยุ่น : การใช้ Environment Variables ช่วยให้โปรแกรมสามารถทำงานในสภาพแวดล้อมที่ต่างกันไปได้อย่างยืดหยุ่น และไม่ติดขัด
- การดูแลรักษา : โดยการใช้ Environment Variables เราสามารถแยกการกำหนดค่าจากโค้ดของแอปพลิเคชัน ทำให้การดูแลรักษาและการปรับปรุงโปรแกรมเป็นไปอย่างมีประสิทธิภาพ



Environment Variables

ใน Node.js เราสามารถเข้าถึง Environment Variables ได้โดยใช้โมดูล `process.env` ซึ่งเป็นอ็อบเจกต์ที่เก็บค่าของ Environment Variables ทั้งหมด โดยที่ `dotenv` เป็นโมดูลที่ช่วยในการโหลด Environment Variables จากไฟล์ `.env` ในโปรเจกต์



Environment Variables



```
npm install dotenv
```



My SQL





My SQL

MySQL เป็นระบบจัดการฐานข้อมูลระบบสัมพันธ์ (RDBMS) แบบ Open-Source ที่ได้รับความนิยม มีความสามารถในการจัดการข้อมูลได้อย่างมีประสิทธิภาพ โดยใช้ภาษา SQL ในการจัดการกับฐานข้อมูลนั้น ๆ การใช้งาน MySQL ในการจัดเก็บข้อมูลในบทความนี้ เราจะจัดการกับฐานข้อมูลโดยดำเนินการผ่านตัว MySQL Shell กับตัว Database ที่อยู่ใน Localhost ของเรา ซึ่งสามารถติดตั้งตัว MySQL และ MySQL Shell



Setup Project



Setup Project



```
npm i dotenv express mysql2 nodemon
```



.env

```
# =====  
# Database configuration  
# =====  
  
DB_HOST = "localhost"      # ที่อยู่ host ของฐานข้อมูล (localhost สำหรับเครื่องตัวเอง)  
DB_USER = "root"           # ชื่อผู้ใช้ MySQL  
DB_PASSWORD = "passw@ed"   # รหัสผ่านผู้ใช้ MySQL  
DB_CONTAINER = "myDatabase" # ชื่อของ Container  
DB_NAME = "myDatabase"     # ชื่อฐานข้อมูลที่จะใช้งาน  
DB_PORT = "3307"           # พอร์ตของ MySQL (ค่า default คือ 3306)
```

<https://goonlinetools.com/snapshot/code/#p0rlpx7xad1llfz7sucgy>



docker-compose.yaml

```
services:
  mysql:
    image: mysql:latest
    container_name: ${DB_CONTAINER}
    environment:
      MYSQL_ROOT_PASSWORD: ${DB_PASSWORD}
      MYSQL_DATABASE: ${DB_NAME}
    ports:
      - "${DB_PORT}:3306"
    volumes:
      - db_data:/var/lib/mysql

volumes:
  db_data:
```

<https://goonlinetools.com/snapshot/code/#8x4xvfmdo5rpeea65zilt>



docker run code



```
docker-compose up -d
```




server.js

```
const express = require('express');
const mysql = require('mysql2');

require('dotenv').config();

const app = express();

const db = mysql.createConnection({
  host: process.env.DB_HOST,      // ชื่อ host ของฐานข้อมูล เช่น localhost
  user: process.env.DB_USER,      // ชื่อผู้ใช้งาน MySQL
  password: process.env.DB_PASSWORD, // รหัสผ่านของผู้ใช้งาน
  database: process.env.DB_NAME,  // ชื่อฐานข้อมูลที่จะใช้
  port: process.env.DB_PORT       // พอร์ตของ MySQL (ค่า default คือ 3306)
});

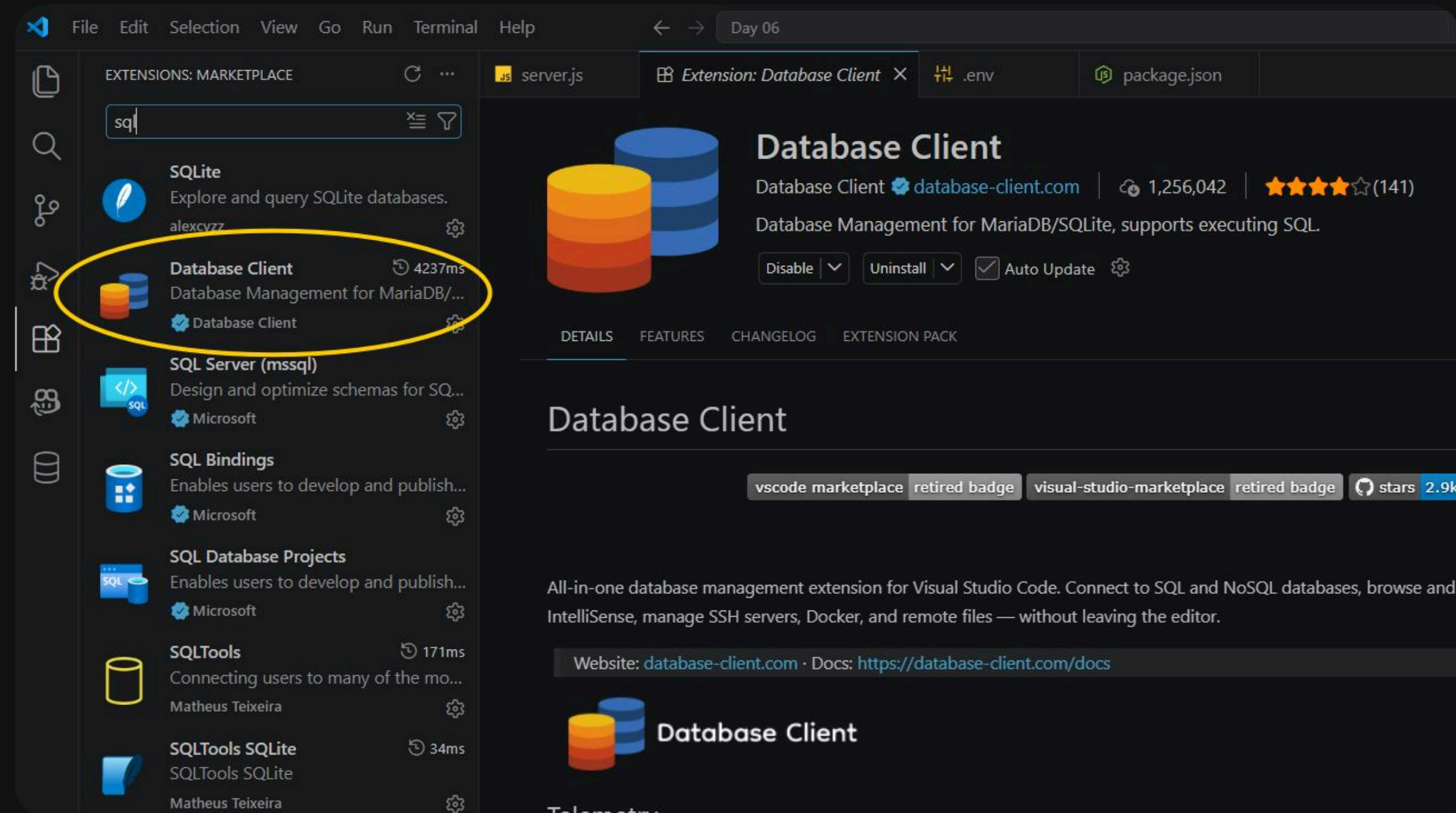
db.connect((err) => {
  if (err) {
    console.log('Error connecting to MySQL database = ', err);
    return;
  }
  console.log('MySQL successfully connected!');
});

app.listen(3000, () => {
  console.log('Server is running on http://localhost:3000');
});
```

<https://goonlinetools.com/snapshot/code/#ggq2evd9t5eh8bsp7rxzu54>



Extension ที่แนะนำ





Setup Database

server.jsusersX.envpackage.json

PropertiesDATAERMockManagermyDatabase

NameusersComment

EngineInnoDBCollateutf8mb4_0900_ai_ciUpdate

DDLColumnForeign KeyIndexTriggerCheck

+C

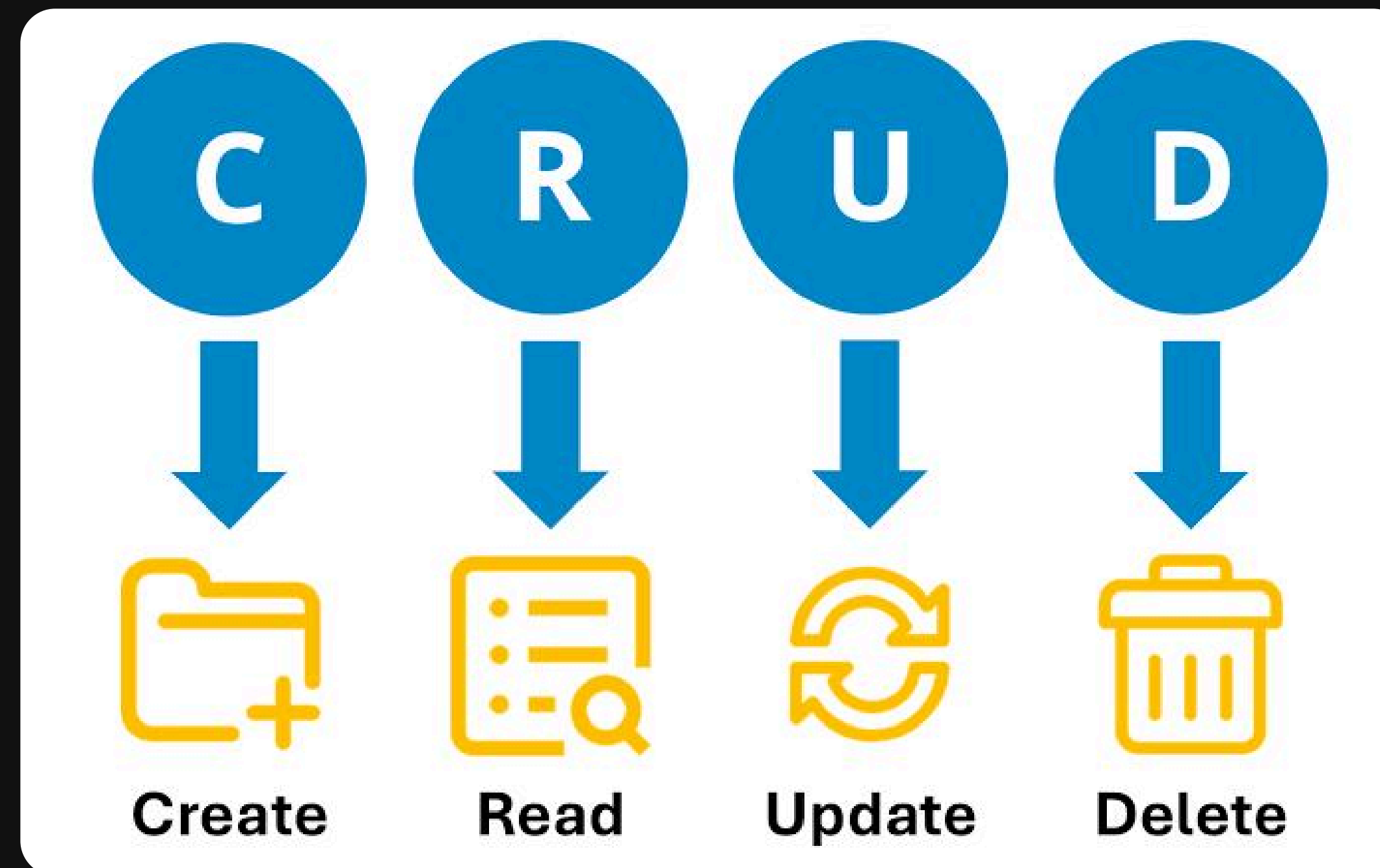
Name	Type	Length	Comment	Default	Not Null	Primary Key	UNIQUE	Auto Increment	Operation
id	int		Primary Key		☒	☒	☒	☒	
email	varchar(255)	255			☐	☐	☐	☐	
fullname	varchar(255)	255			☐	☐	☐	☐	
password	varchar(255)	255			☐	☐	☐	☐	



CRUD



CRUD





CRUD

CRUD คือ แนวคิดพื้นฐานที่ใช้ในการเขียนโปรแกรม หรือเว็บแอปพลิเคชัน เพื่อบ่งบอกลักษณะการกระทำ (เพิ่ม อ่าน แก้ไข ลบ) ข้อมูล

- C (Create) = PUT / POST
- R (Read) = GET
- U (Update) = PUT / POST / PATCH
- D (Delete) = DELETE

ต่างกันอย่างไร? ...เลือกใช้ให้ถูกต้อง!

GET VS POST

METHOD METHOD

GET	POST
https://www.borntodev.com/GetExample/?no=813	https://www.borntodev.com/PostExample/
มีการแสดงข้อมูลการร้องขอบน URL และจะไม่มี Request Body	ไม่มีการแสดงข้อมูลการร้องขอบน URL แต่จะแสดงใน Request Body
มีการเก็บแคช (Cache)	ไม่มีการเก็บแคช (Cache)
เก็บ History และ Bookmark ได้	เก็บ History และ Bookmark ไม่ได้
มีข้อจำกัดที่เกิดจากความยาว URL	ไม่มีข้อจำกัดที่เกิดจาก URL
HTTP Method สำหรับร้องขอข้อมูล ใช้กับข้อมูลที่สามารถให้ผู้ใช้มองเห็นได้ เช่น ข้อมูลทั่วไปมาสร้างเป็นหน้าเว็บ	HTTP Method สำหรับร้องขอข้อมูล ใช้กับข้อมูลที่ไม่สามารถให้ผู้ใช้มองเห็นได้ เช่น รหัสผ่าน การเขียนข้อมูลในฐานข้อมูล

borntoDev

PUT กับ PATCH

ต่างกันยังไง?

ไขข้อข้องใจ HTTP Method ที่เอาไว้อัปเดตข้อมูล

PUT	PATCH
อัปเดตทั้ง resource	อัปเดตเฉพาะส่วนที่เปลี่ยนแปลง
ส่งข้อมูล resource ทั้งหมด	ส่งเฉพาะข้อมูลที่ต้องการเปลี่ยนแปลง
เซิร์ฟเวอร์จะลบ resource เดิม และสร้างใหม่ด้วยข้อมูลใหม่	เซิร์ฟเวอร์จะเปลี่ยนแปลงเฉพาะส่วนที่ระบุ
ไม่เหมาะสมสำหรับการอัปเดตเพียงบางส่วนของ resource	เหมาะสมมากสำหรับการอัปเดตเพียงบางส่วนของ resource
เหมาะสำหรับการอัปเดต resource ทั้งหมด เช่น การเปลี่ยนโปรไฟล์ผู้ใช้	เหมาะสำหรับการอัปเดตบางส่วน เช่น การเปลี่ยนชื่อหรืออีเมล

EXAMPLE

PUT	PATCH
<pre>PUT /users/1 Content-Type: application/json { "name": "John Doe", "email": "john.doe@example.com" }</pre>	<pre>PATCH /users/1 Content-Type: application/json { "email": "john.doe@example.com" }</pre>

BORNTO DEV



Postman



<https://www.postman.com/postman>



Postman

Postman เป็นเครื่องมือที่ไว้ใช้สำหรับ API Developers ซึ่งหากให้อธิบายในความหมายง่ายๆก็คือ โดยปกติแล้วเมื่อเราเขียน API Service ขึ้นมาเรามักจะใช้ Postman ที่เป็น API Testing tool ในการส่ง Request และดู Response ที่ได้ต่างๆกลับมา



เขียน Create routes ขึ้น

```
// CREATE Routes
app.post("/create", async (req, res) => {
  const { email, name, password } = req.body;

  try {
    db.query(
      "INSERT INTO users(email, fullname, password) VALUES(?, ?, ?)",
      [email, name, password],
      (err, results, fields) => {
        if (err) {
          console.log("Error while inserting a user into the database", err);
          return res.status(400).send();
        }
        return res.status(201).json({ message: "New user successfully created!" });
      }
    )
  } catch (err) {
    console.log(err);
    return res.status(500).send();
  }
})
```

<https://goonlinetools.com/snapshot/code/#h4u417cfhvqa7rmrsu949>



express.json()

`express.json()` คือฟังก์ชัน Middleware ในเฟรมเวิร์ก Express.js ของ Node.js ทำหน้าที่แปลงข้อมูล ประเภท JSON ที่ส่งมาจากฝั่งผู้ใช้งาน (Client) ในส่วนของ Request Body ให้กลายเป็นอ็อบเจกต์ JavaScript อัตโนมัติ เพื่อให้เซิร์ฟเวอร์นำข้อมูลไปใช้งานต่อได้ทันทีผ่านตัวแปร `req.body`



Setup `express.json()`



```
app.use(express.json());
```

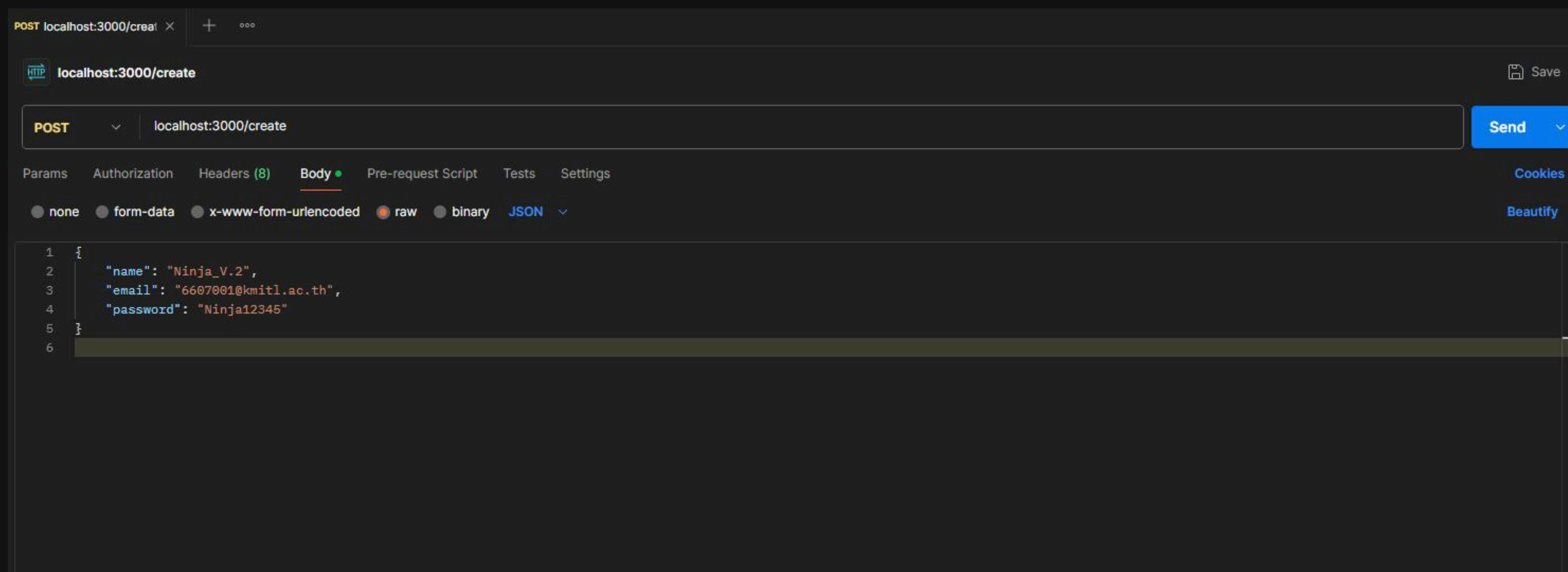


เพิ่มบรรทัดนี้เข้าไปด้วย

```
Help    ← →    Day 06
server.js  X  users  .env
server.js > app.post("/create") callback
1  const express = require('express');
2  const mysql = require('mysql2');
3
4  require('dotenv').config();
5
6  const app = express();
7  app.use(express.json());
8
```




Test ผ่าน Postman





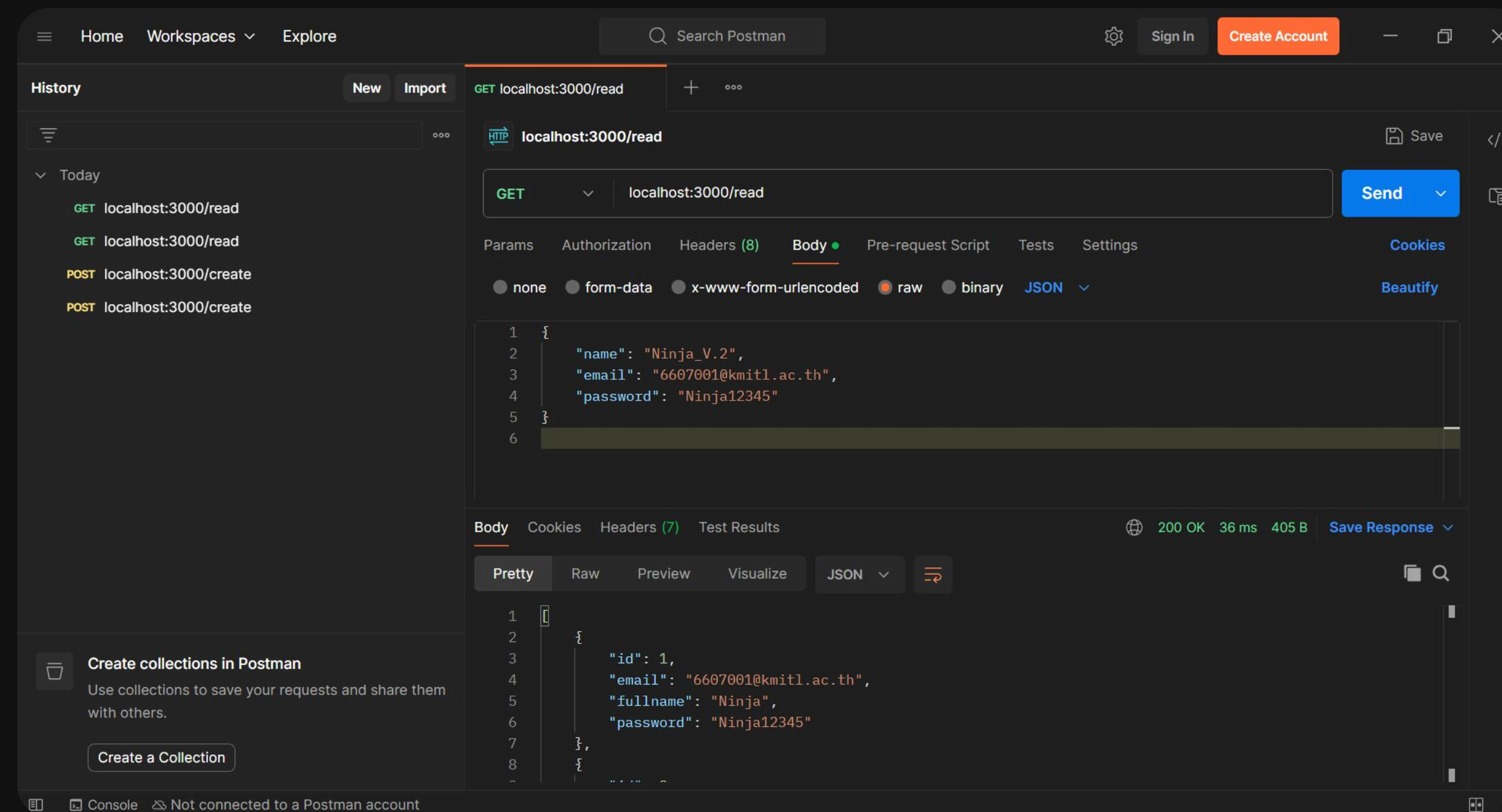
เพิ่ม Read

```
// READ
app.get("/read", async (req, res) => {
  try {
    db.query("SELECT * FROM users", (err, results, fields) => {
      if (err) {
        console.log(err);
        return res.status(400).send();
      }
      res.status(200).json(results)
    })
  } catch(err) {
    console.log(err);
    return res.status(500).send();
  }
});
```

<https://goonlinetools.com/snapshot/code/#tb0sl318c8f420lkwhchh>



Test ผ่าน Postman





เพิ่ม Read single user

```
// READ single users from db
app.get("/read/single/:email", async (req, res) => {
  const email = req.params.email;

  try {
    db.query("SELECT * FROM users WHERE email = ?", [email], (err, results, fields) => {
      if (err) {
        console.log(err);
        return res.status(400).send();
      }
      res.status(200).json(results)
    })
  } catch (err) {
    console.log(err);
    return res.status(500).send();
  }
});
```

<https://goonlinetools.com/snapshot/code/#mz3dqx9tqgrbg67d1x541k>



ลอง Test ผ่าน Postman อีกครั้ง



เพิ่ม Update data

```
// UPDATE data
app.patch("/update/:email", async (req, res) => {
  const email = req.params.email;
  const newPassword = req.body.newPassword;

  try {
    db.query("UPDATE users SET password = ? WHERE email = ?", [newPassword, email], (err, results, fields) => {
      if (err) {
        console.log(err);
        return res.status(400).send();
      }
      res.status(200).json({ message: "User password updated successfully!" });
    })
  } catch (err) {
    console.log(err);
    return res.status(500).send();
  }
});
```

<https://goonlinetools.com/snapshot/code/#p52n92w0pgn3mps6fj6r>



ลอง Test ผ่าน Postman อีกครั้ง



เพิ่ม Delete

```
// DELETE
app.delete("/delete/:email", async (req, res) => {
  const email = req.params.email;

  try {
    db.query("DELETE FROM users WHERE email = ?", [email], (err, results, fields) => {
      if (err) {
        console.log(err);
        return res.status(400).send();
      }
      if (results.affectedRows === 0) {
        return res.status(404).json({ message: "No user with that email!" });
      }
      return res.status(200).json({ message: "User deleted successfully!" });
    })
  } catch (err) {
    console.log(err);
    return res.status(500).send();
  }
});
```

<https://goonlinetools.com/snapshot/code/#nn8koa3n56kcmbmyqvvhg>



ลอง Test ผ่าน Postman อีกครั้ง



Soft Delete



Soft Delete

สอนน้อง !
มาทำความรู้จัก
SOFT DELETE

เมื่อการ DELETE ไม่ได้ลบให้หายไป !

เคยมี ?
ลบข้อมูลไปแล้ว
อยากได้คืน



เรามีวิธีที่จะทำให้คุณ "ลบแล้วไม่หาย"
มาทำความรู้จักกับ **SOFT DELETE** กัน !

Soft Delete

รู้จัก **SOFT DELETE** ใน 10 วินาที



ถ้าการลบปกติคือการ "ทิ้งขยะ"
SOFT DELETE ก็คือการ "**ซ่อนของ**" นั่นเอง !

เราไม่ได้ลบข้อมูลออกจริงๆ แต่แค่ **ทำเครื่องหมาย** ว่า
"อ้อ... อันนี้ถูกลบไปแล้วนะ"



• แล้วมันทำงานยังไง ?

ง่ายมาก ! แทนที่จะลบทิ้ง เราแค่เพิ่มข้อมูลว่า...

"ลบเมื่อไหร่" (deleted_at) หรือ "ถูกลบรึยัง" (is_deleted)

เหมือนเราแปะป้าย "ของเก่า" ไว้ แต่ยังไม่ทิ้งของชิ้นนั้นไปไหน !

• มีประโยชน์ยังไง ?

นึกภาพว่าคุณมีร้านค้าออนไลน์



ข้อมูลไม่หาย ความสัมพันธ์ไม่พัง
ตรวจสอบย้อนหลังได้ ปลอดภัยกว่าเยอะ !





Soft Delete



ลองดูโค้ดกันหน่อย !

แต่ก่อน

```
DELETE FROM USERS WHERE ID = 1;
```

(ลบทิ้งเลย ไม่เหลือซาก ☠)

ตอนนี้

```
UPDATE USERS  
SET DELETED_AT = NOW()  
WHERE ID = 1;
```

(แค่แปะป้าย "ลบแล้ว" พร้อมเวลา 🕒)



สิ่งที่ต้องระวัง !

พื้นที่เก็บข้อมูลจะเพิ่มขึ้น
เพราะเราไม่ได้ลบจริง

การค้นหาอาจช้าลงนิดหน่อย
และต้องจัดการเรื่องข้อมูลซ้ำให้ดี



แต่ไม่ต้องกังวล! มีวิธีการจัดการ
ที่จะเล่าในหน้าถัดไป... 🗨



Soft Delete



เทคนิคดี ๆ จัดการ **SOFT DELETE**



- ตั้งเวลาลบข้อมูลเก่าอัตโนมัติ
- เพิ่ม **index** ให้ค้นหาเร็วขึ้น
- ใช้ **composite unique key** ป้องกันข้อมูลซ้ำ

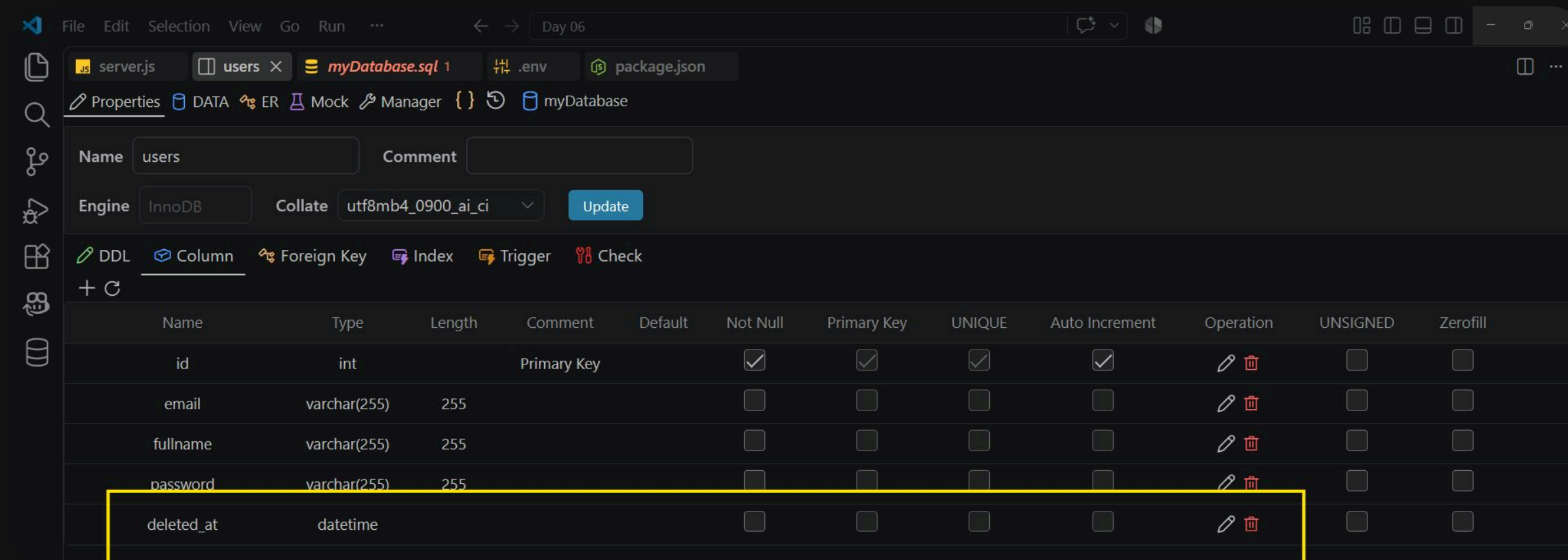
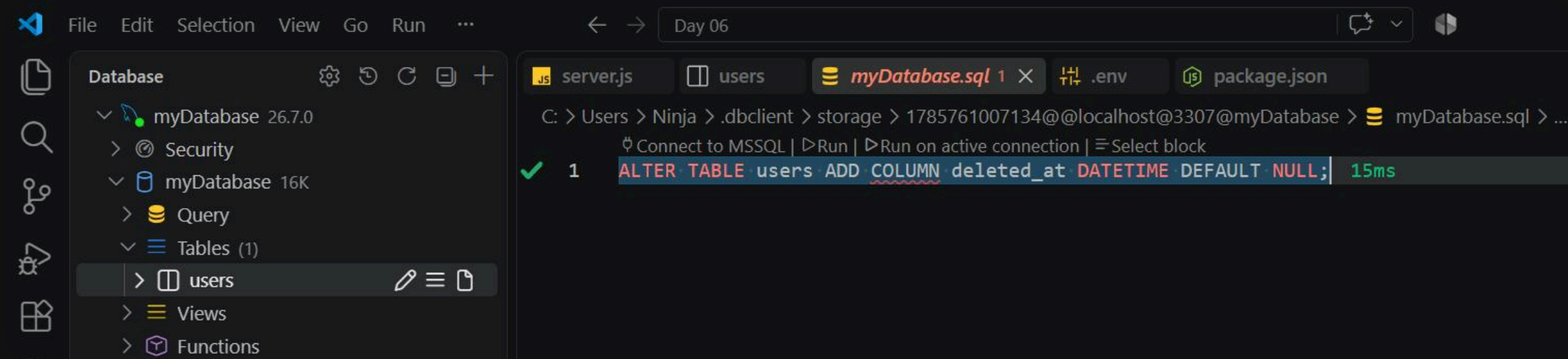
ทำแค่นี้ ระบบคุณก็จะแจ่มเวอร์ !



มาใส่ Soft Delete กัน



เพิ่มคอลัมน์ deleted_at ลงในตาราง users





ปรี๊ดป๊อง Code มี 3 จุดใหญ่ ๆ



แก้ Read กับ Read single users

```
// READ (ดึงเฉพาะข้อมูลที่ยังไม่ถูก Soft Delete)
app.get("/read", async (req, res) => {
  try {
    // เพิ่มเงื่อนไข WHERE deleted_at IS NULL
    db.query("SELECT * FROM users WHERE deleted_at IS NULL", (err, results, fields) => {
      if (err) {
        console.log(err);
        return res.status(400).send();
      }
      res.status(200).json(results);
    });
  } catch (err) {
    console.log(err);
    return res.status(500).send();
  }
});
```

```
// READ single users from db (ดึงเฉพาะข้อมูลที่ยังไม่ถูก Soft Delete)
app.get("/read/single/:email", async (req, res) => {
  const email = req.params.email;

  try {
    // เพิ่มเงื่อนไข WHERE email = ? AND deleted_at IS NULL
    db.query("SELECT * FROM users WHERE email = ? AND deleted_at IS NULL", [email], (err, results, fields) => {
      if (err) {
        console.log(err);
        return res.status(400).send();
      }
      if (results.length === 0) {
        return res.status(404).json({ message: "User not found or deleted" });
      }
      res.status(200).json(results[0]);
    });
  } catch (err) {
    console.log(err);
    return res.status(500).send();
  }
});
```

<https://goonlinetools.com/snapshot/code/#jzyowcupqy2scbfxqts2i>



แก้ไข Update

```
// UPDATE data (อัปเดตเฉพาะบัญชีที่ยังไม่ถูกลบ)
app.patch("/update/:email", async (req, res) => {
  const email = req.params.email;
  const newPassword = req.body.newPassword;

  try {
    // เพิ่มเงื่อนไข AND deleted_at IS NULL
    db.query("UPDATE users SET password = ? WHERE email = ? AND deleted_at IS NULL", [newPassword, email], (err, results, fields) => {
      if (err) {
        console.log(err);
        return res.status(400).send();
      }
      if (results.affectedRows === 0) {
        return res.status(404).json({ message: "User not found or deleted" });
      }
      res.status(200).json({ message: "User password updated successfully!" });
    });
  } catch (err) {
    console.log(err);
    return res.status(500).send();
  }
});
```

<https://goonlinetools.com/snapshot/code/#0dck04iggqddbr1y8c7lv>



แก้ Delete เป็นแบบ Soft Delete

```
// SOFT DELETE (เปลี่ยนจากการทำ DELETE เป็น UPDATE เวลาปัจจุบันลง deleted_at)
app.delete("/delete/:email", async (req, res) => {
  const email = req.params.email;

  try {
    // อัปเดตคอลัมน์ deleted_at ด้วยเวลาปัจจุบัน (NOW()) เฉพาะรายการที่ยังไม่ได้ลบ
    db.query(
      "UPDATE users SET deleted_at = NOW() WHERE email = ? AND deleted_at IS NULL",
      [email],
      (err, results, fields) => {
        if (err) {
          console.log(err);
          return res.status(400).send();
        }
        if (results.affectedRows === 0) {
          return res.status(404).json({ message: "No active user found with that email!" });
        }
        return res.status(200).json({ message: "User soft-deleted successfully!" });
      }
    );
  } catch (err) {
    console.log(err);
    return res.status(500).send();
  }
});
```

<https://goonlinetools.com/snapshot/code/#hfulrhx5bh50lkvb15iqvjpp>



กลับไป Test บน Postman กัน



No SQL





No SQL

NoSQL (Not Only SQL) คือระบบจัดการฐานข้อมูล (Database Management System) รูปแบบหนึ่งที่ไม่ได้ใช้ตาราง (Table) ที่มีโครงสร้างตายตัว เหมือนกับ SQL (Relational Database) แบบดั้งเดิม

NoSQL ถูกออกแบบมาเพื่อรองรับข้อมูลยุคใหม่ที่มีปริมาณมหาศาล (Big Data) มีการเปลี่ยนแปลงโครงสร้างบ่อย และต้องการความเร็วสูงในการอ่าน/เขียนข้อมูล



จุดเด่นสำคัญของ NoSQL

- Schema-less (ไม่มีโครงสร้างตายตัว): ไม่จำเป็นต้องกำหนดคอลัมน์ล่วงหน้า ข้อมูลแต่ละ Row (หรือ Document) ไม่จำเป็นต้องมี Field เหมือนกัน
- Horizontal Scaling (ขยายระบบตามแนวนอนได้ง่าย): สามารถเพิ่มเครื่อง Server เข้ามาช่วยกันทำงานได้ง่ายกว่า SQL
- High Performance: ออกแบบมาให้อ่านและเขียนข้อมูลได้เร็วมาก เหมาะกับแอปพลิเคชันที่รองรับผู้ใช้งานพร้อมกันจำนวนมาก



ประเภทของ NoSQL

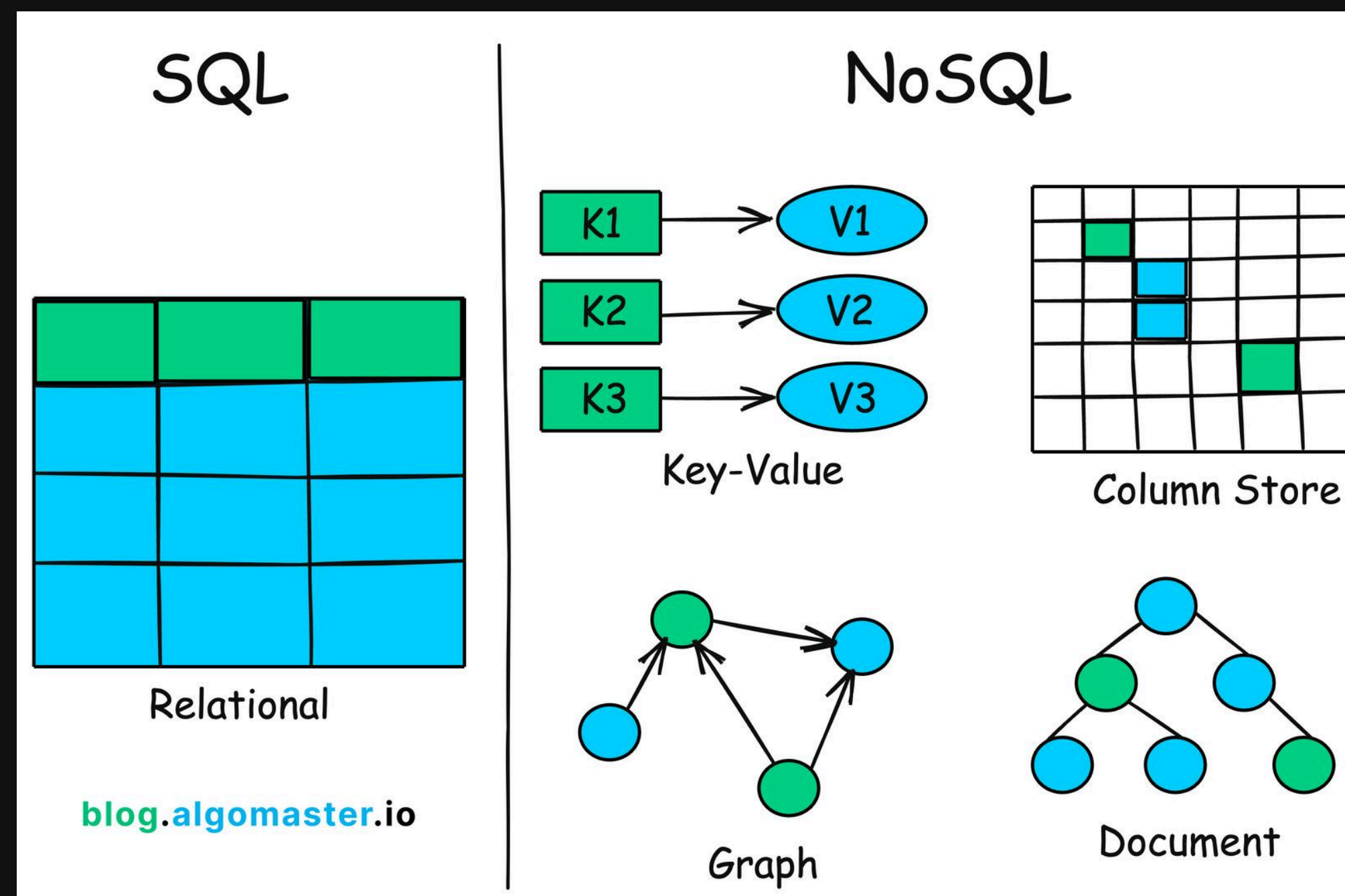
NoSQL แบ่งออกเป็น 4 ประเภทหลักๆ ตามรูปแบบการจัดเก็บ

- Document Database - เก็บข้อมูลในรูปแบบ Document (ส่วนใหญ่เป็น JSON หรือ BSON)
- Key-Value Store - เก็บข้อมูลเป็นคู่ Key (รหัส) และ Value (ค่า) เหมือนพจนานุกรม
- Wide-Column Store (Column-Family) - เก็บข้อมูลแยกตาม Column แทนที่จะเป็น Row
- Graph Database - เก็บข้อมูลในรูปของ Node (โหนด) และ Edge (เส้นเชื่อมความสัมพันธ์)



SQL และ NoSQL ต่างกันยังไง ?

โครงสร้างและการเก็บข้อมูล



- SQL : เก็บเป็นตาราง แถว และคอลัมน์ เหมือนตารางในโปรแกรม Excel ต้องกำหนดรูปแบบข้อมูล (Schema) ไว้ล่วงหน้า
- NoSQL : เก็บเป็นรูปแบบอื่น เช่น Document (JSON), Key-Value หรือ Column-family ปรับเปลี่ยนโครงสร้างข้อมูลได้ตลอดเวลา



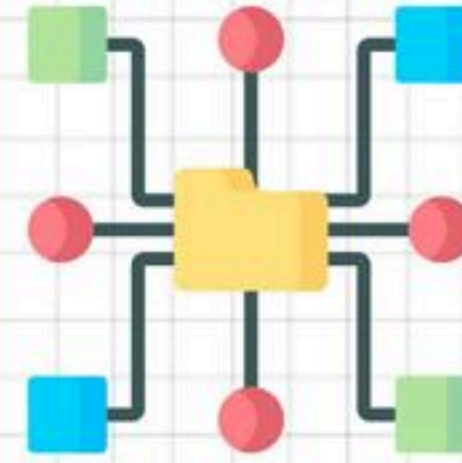
การขยายระบบและการใช้งาน

- SQL: เหมาะกับงานที่ต้องการความถูกต้องแม่นยำสูง เช่น ระบบธนาคารหรือบัญชี นิยมขยายเครื่องโดยเพิ่มพลังของเครื่องเดิม (Vertical Scaling)
- NoSQL: เหมาะกับข้อมูลปริมาณมากและต้องการความเร็วสูง เช่น ระบบแชท, ข้อมูล IoT นิยมขยายเครื่องโดยเพิ่มจำนวนเครื่อง (Horizontal Scaling)

BORNTO
DEV

ทำไม JSON ถึง**เหมาะ**กับ **NoSQL** กันนะ ?

มาดู 3 เหตุผลสำคัญที่ทำให้ข้อมูลแบบ JSON นิยมใช้ในฐานข้อมูลแบบ NoSQL

เก็บข้อมูลได้ยืดหยุ่น  จัดเก็บข้อมูลง่าย ๆ แบบไม่ต้องใช้โครงสร้างตาราง (แถวและคอลัมน์) เก็บข้อมูลได้ไม่จำกัด และขยายพื้นที่ได้ง่าย	เข้าถึงข้อมูลได้ง่าย  JSON นั้นอ่านและเขียนข้อมูลได้ง่าย รองรับข้อมูลได้หลายประเภท เช่น Text, Numeric, Lists, และ Key-value	โครงสร้างยืดหยุ่น  จัดการง่ายกว่าโครงสร้าง Schema ใน RDMS เพิ่มและแก้ไขข้อมูลได้ง่าย ปรับเปลี่ยนโครงสร้างได้อย่างยืดหยุ่น ทั้งยังทำได้อย่างรวดเร็วด้วย
---	--	---





MongoDB



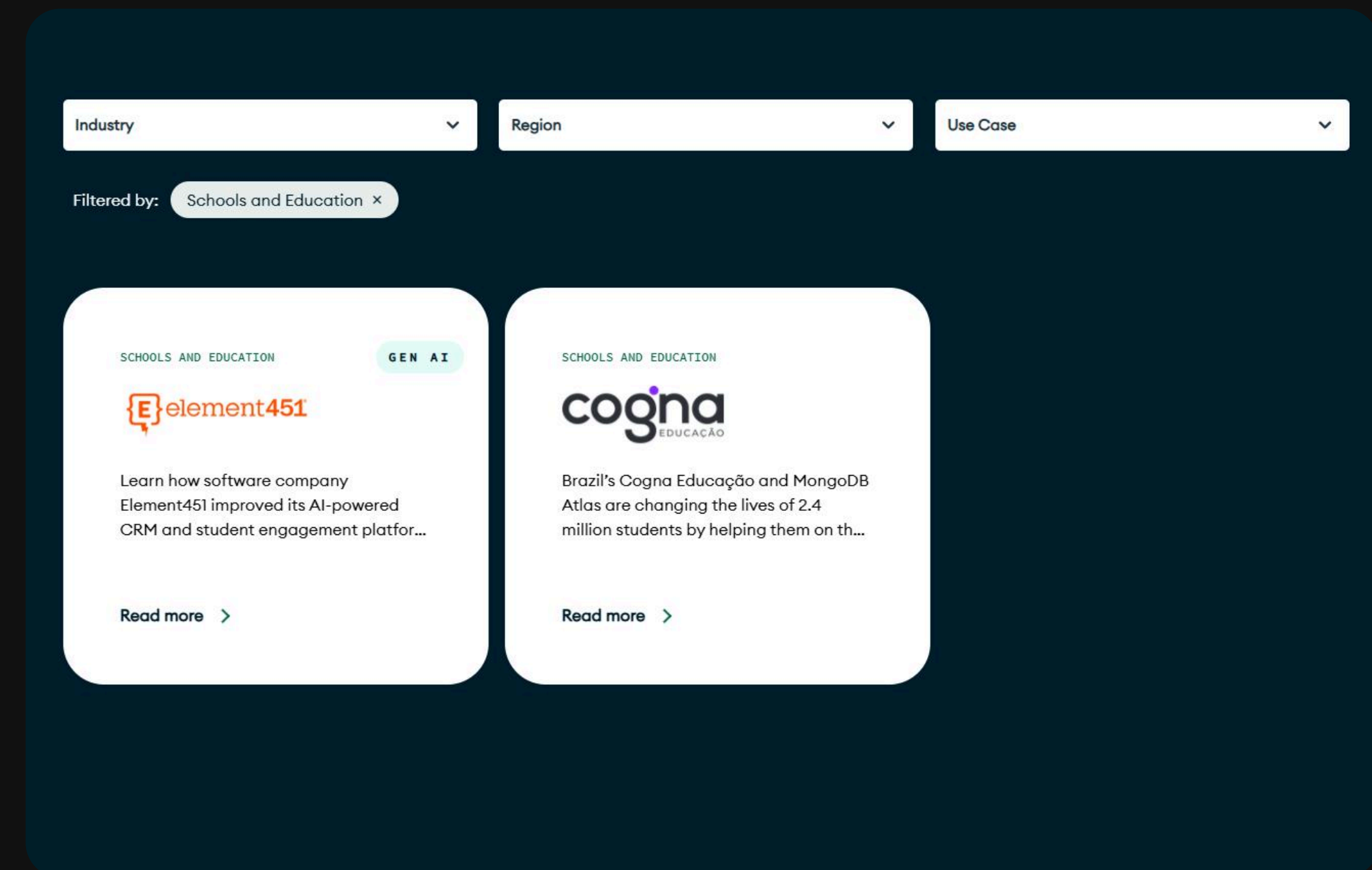
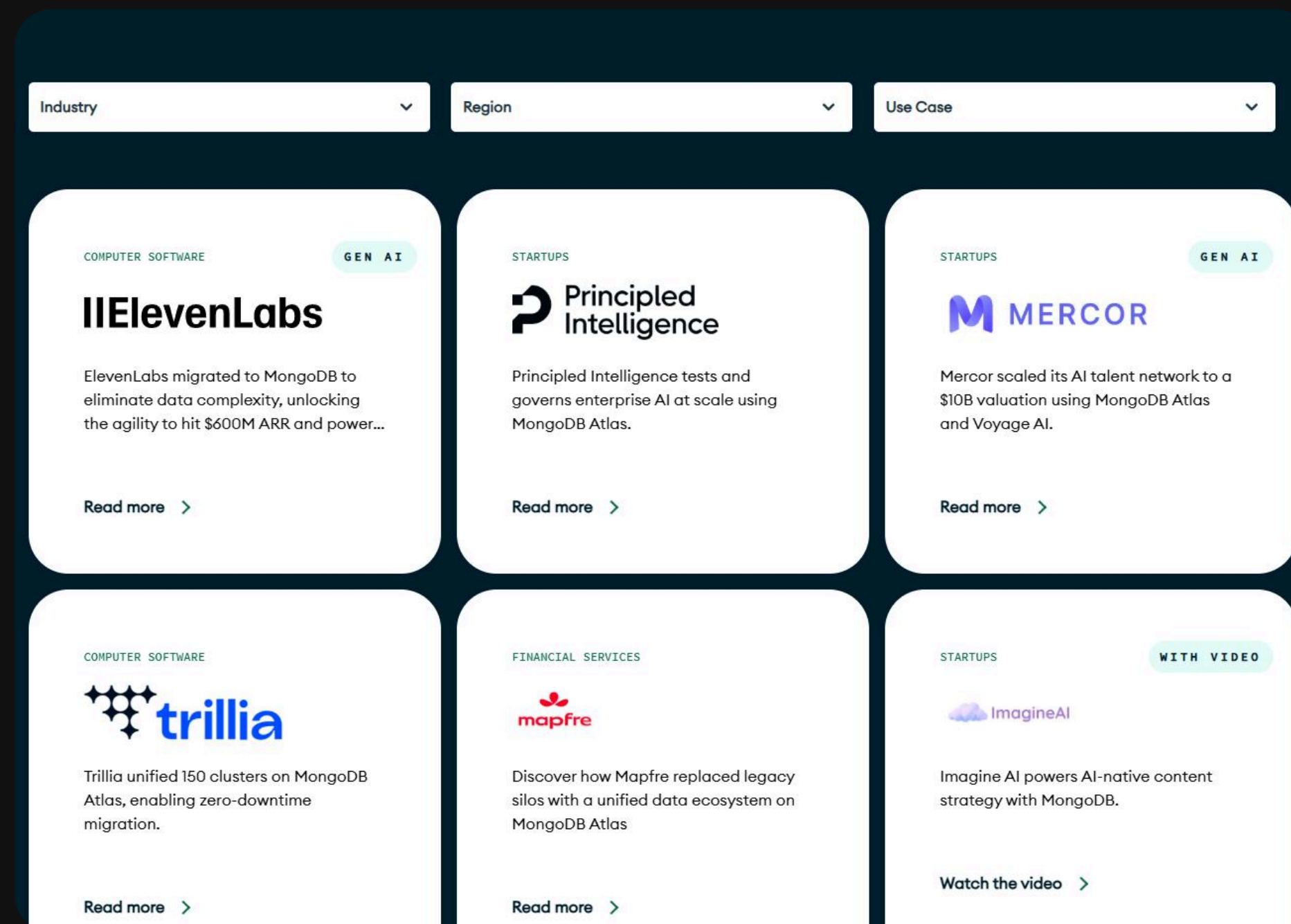


MongoDB

MongoDB คือระบบจัดการฐานข้อมูลประเภท NoSQL (Not only SQL) ที่จัดเก็บข้อมูลในรูปแบบเอกสาร (Document-based) คล้ายกับไฟล์ JSON ทำให้มีความยืดหยุ่นสูง ไม่ต้องกำหนดโครงสร้างตาราง (Schema) ล่วงหน้าเหมือนฐานข้อมูลแบบ SQL ทั่วไป



MongoDB สามารถนำไปใช้ทำอะไรได้บ้างในธุรกิจ



<https://www.mongodb.com/solutions/customer-case-studies>



เปรียบเทียบศัพท์ SQL vs MongoDB

SQL (Relational DB)	MongoDB	คำอธิบาย
Database	Database	ฐานข้อมูล
Table	Collection	กลุ่มของข้อมูลประเภทเดียวกัน
Row / Record	Document	ข้อมูล 1 รายการ (รูปทรง JSON)
Column	Field	หัวข้อ/คุณลักษณะของข้อมูล
Primary Key (id)	_id	Key หลักสร้างให้อัตโนมัติ (ObjectId)



MongoDB Community Server



MONGODB COMMUNITY SERVER

MongoDB Community Server Download

Community Server

The Community version of our distributed database offers a flexible document data model along with support for:

- Ad-hoc queries
- Secondary indexing
- Real-time aggregations for powerful ways to access and analyze your data

Select Package

Or Try MongoDB Atlas

Try our fully managed service, MongoDB Atlas, for free with capabilities designed to support you from prototype to production, including:

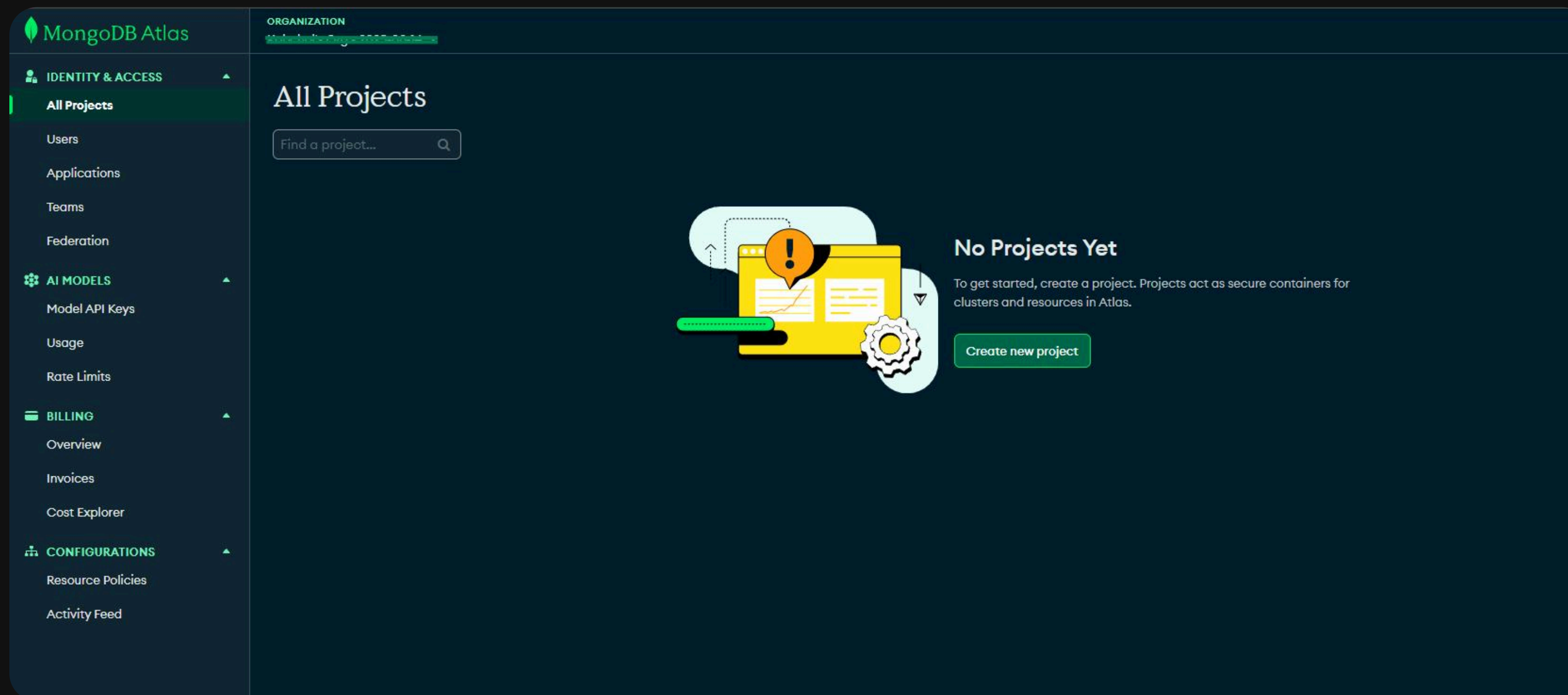
- Auto-scaling
- High availability by default
- Automatic backups
- Integrated full-text search, vector search, and stream processing
- Multi-region and multi-cloud support

Get Started For Free

MongoDB Community Server คือ เวอร์ชันฟรี (Open Source / Source-Available) ของ MongoDB ที่เปิดให้ทุกคนดาวน์โหลดมาติดตั้งและรันใช้งานได้เองบนเครื่องคอมพิวเตอร์ของคุณเอง หรือบน Server/VPS ของบริษัท ถือเป็นเวอร์ชันมาตรฐานที่นักพัฒนา (Developers) และองค์กรส่วนใหญ่นิยมใช้มากที่สุด



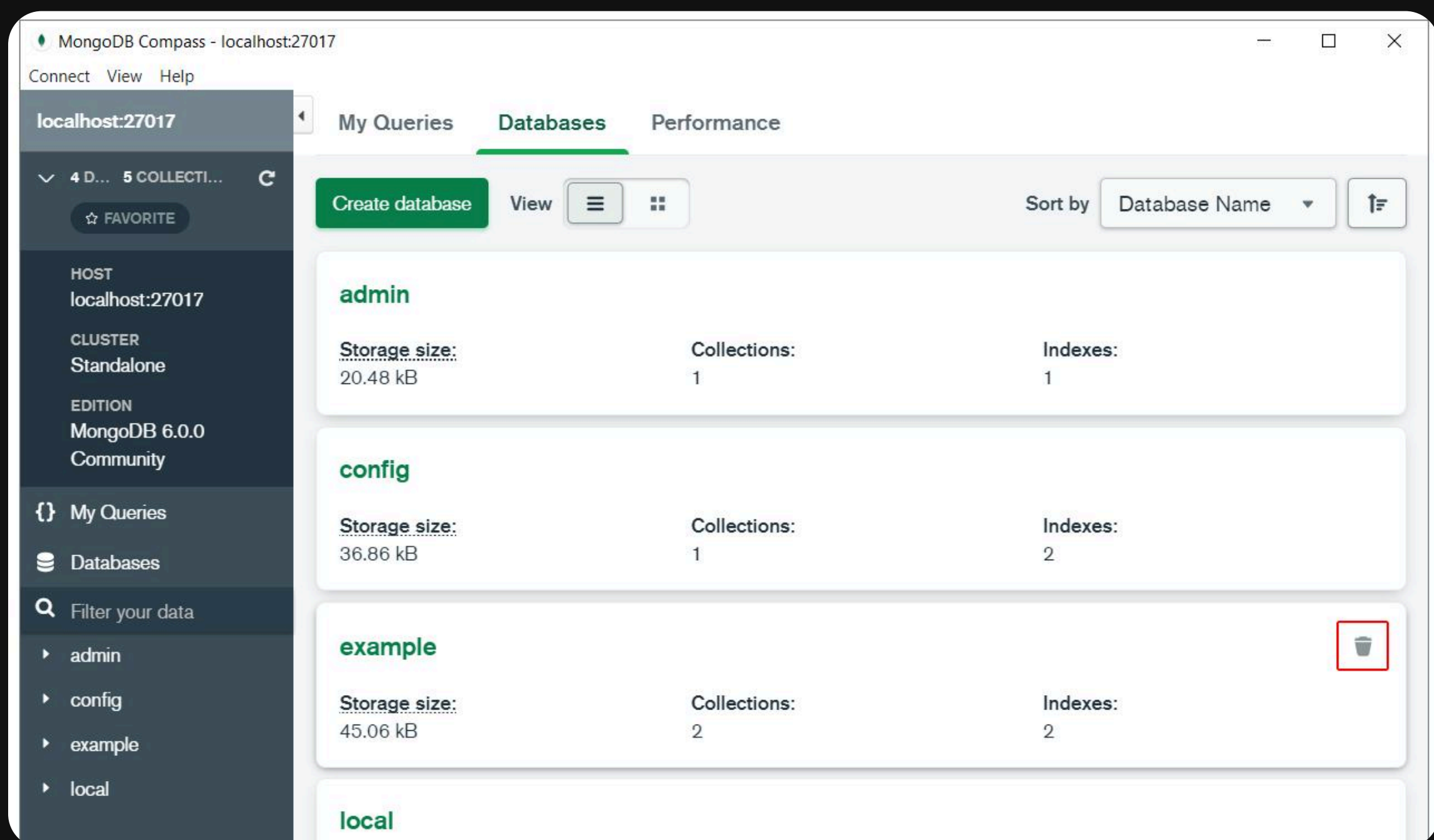
Mongodb Atlas



MongoDB Atlas คือ
บริการฐานข้อมูลบนคลาวด์แบบจัดการเต็มรูปแบบ (Fully Managed Database-as-a-Service หรือ DBaaS)
ของระบบฐานข้อมูล NoSQL
ยอดนิยมอย่าง MongoDB
ช่วยให้เราใช้งานฐานข้อมูลได้ทันทีบนคลาวด์ชั้นนำอย่าง AWS, Google Cloud และ Azure โดยไม่ต้องตั้งค่าเซิร์ฟเวอร์เอง



Mongodb Compass

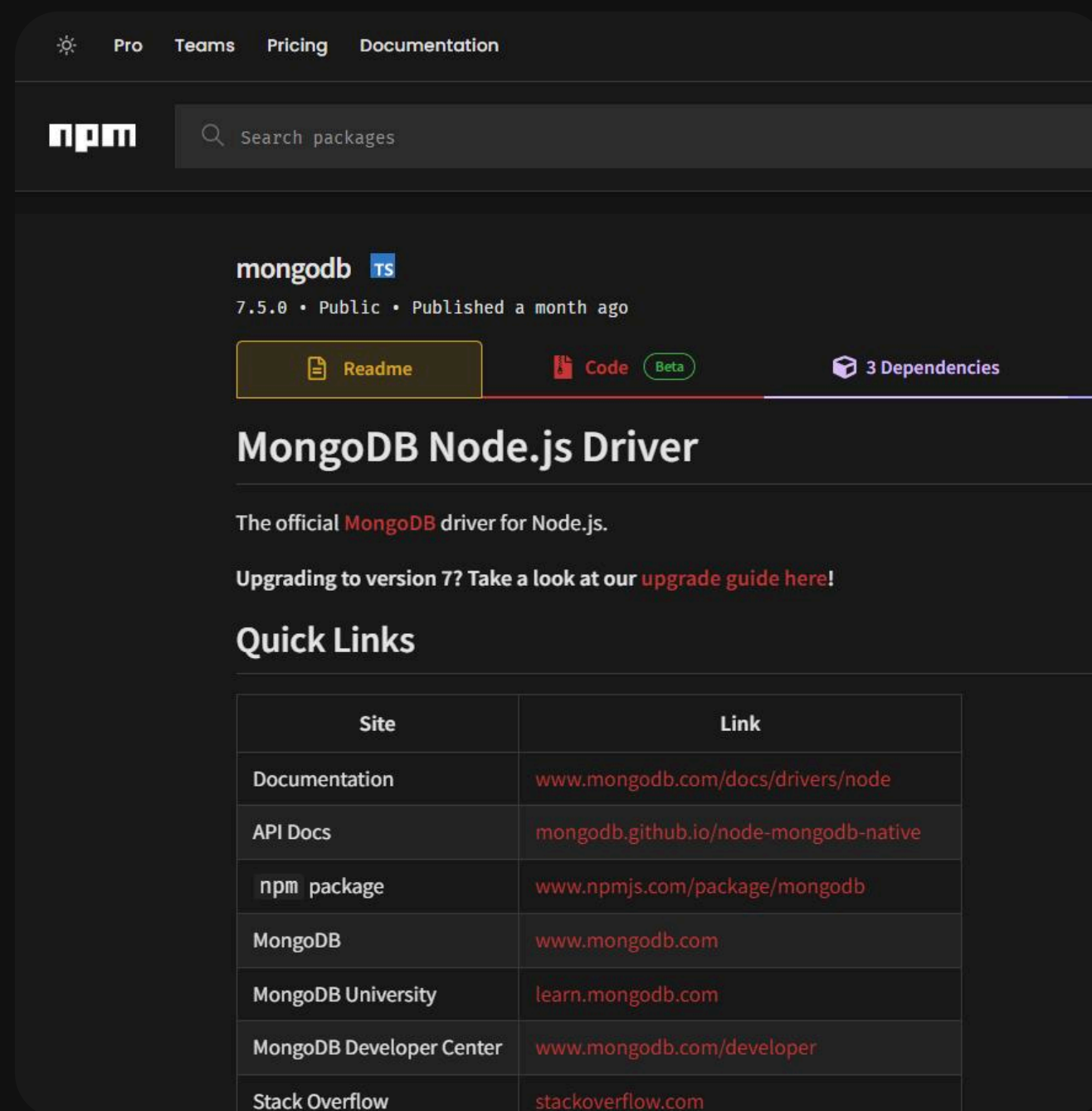


MongoDB Compass คือโปรแกรมประเภท GUI (Graphical User Interface)

ที่ออกแบบมาเพื่อช่วยให้ผู้ใช้งานสามารถมองเห็น สำรวจ และจัดการข้อมูลในฐานข้อมูล MongoDB ผ่านหน้าจอภาพที่ใช้งานง่าย โดยไม่ต้องพิมพ์คำสั่งยาว ๆ ในคอมมานด์ไลน์



MongoDB Drivers



MongoDB Drivers คือ Library หรือ Code สำเร็จรูป ที่ทำหน้าที่เป็น "สะพานเชื่อมต่อ" (Bridge) และตัวกลางในการสื่อสารระหว่าง แอปพลิเคชันที่เราเขียน (เช่น Node.js, Python, Java) กับ MongoDB Database Server

พูดง่ายๆ คือ ตัว MongoDB Server มันคุยภาษาของมันเอง ถ้าเราเขียนโค้ดภาษา JavaScript (Express.js) แล้วอยากสั่งให้ MongoDB บันทึกข้อมูล เราต้องใช้ MongoDB Driver สำหรับ Node.js แปลคำสั่ง JavaScript ให้กลายเป็นภาษาที่ MongoDB เข้าใจนั่นเอง



Data Hierarchy ของ MongoDB

เมื่อเราเขียน Code และทำงานกับข้อมูลใน MongoDB ข้อมูลจะถูกจัดระเบียบตามลำดับขั้นดังนี้

- Database (ฐานข้อมูล): เป็นถังเก็บข้อมูลระดับสูงสุด ในหนึ่งเซิร์ฟเวอร์หรือ Cluster สามารถมี Database ได้หลายตัว แต่ละ Database จะแยกชุดข้อมูลออกจากกันอย่างอิสระ
- Collection (คอลเลกชัน): เป็นกลุ่มของ Document ที่อยู่ใน Database เดียวกัน เทียบได้กับ Table (ตาราง) ในระบบฐานข้อมูลแบบ Relational โดย Collection ไม่บังคับว่า Document ข้างในต้องมีโครงสร้างเหมือนกันเป๊ะ
- Document (เอกสาร): เป็นหน่วยย่อยของข้อมูลที่เก็บจริง อยู่ในรูปแบบฟอร์แมต JSON/BSON (Key-Value Pairs) เทียบได้กับ Record หรือ Row (แถว) ในฐานข้อมูลทั่วไป
- Field (ฟิลด์): เป็นคู่ของชื่อและค่า (Key-Value) ที่อยู่ภายใน Document เทียบได้กับ Column (คอลัมน์) ในตาราง



MongoDB



```
npm i mongoose
```



Setup Project



เพิ่มไปใน .env

```
● ● ●  
  
# =====  
# MongoDB configuration  
# =====  
MONGODB_URI = ""
```

<https://goonlinetools.com/snapshot/code/#xg03zltpq6ji2c7cc3uum>



server_mongodb.js

```
const express = require('express');
const mongoose = require('mongoose');

require('dotenv').config();

const app = express();
app.use(express.json());

// เชื่อมต่อ MongoDB
mongoose.connect(process.env.MONGODB_URI) // URL ของ MONGODB
  .then(() => console.log('mongodb successfully connected!'))
  .catch((err) => console.error(err));

app.listen(3000, () => {
  console.log('Server is running on http://localhost:3000');
});
```

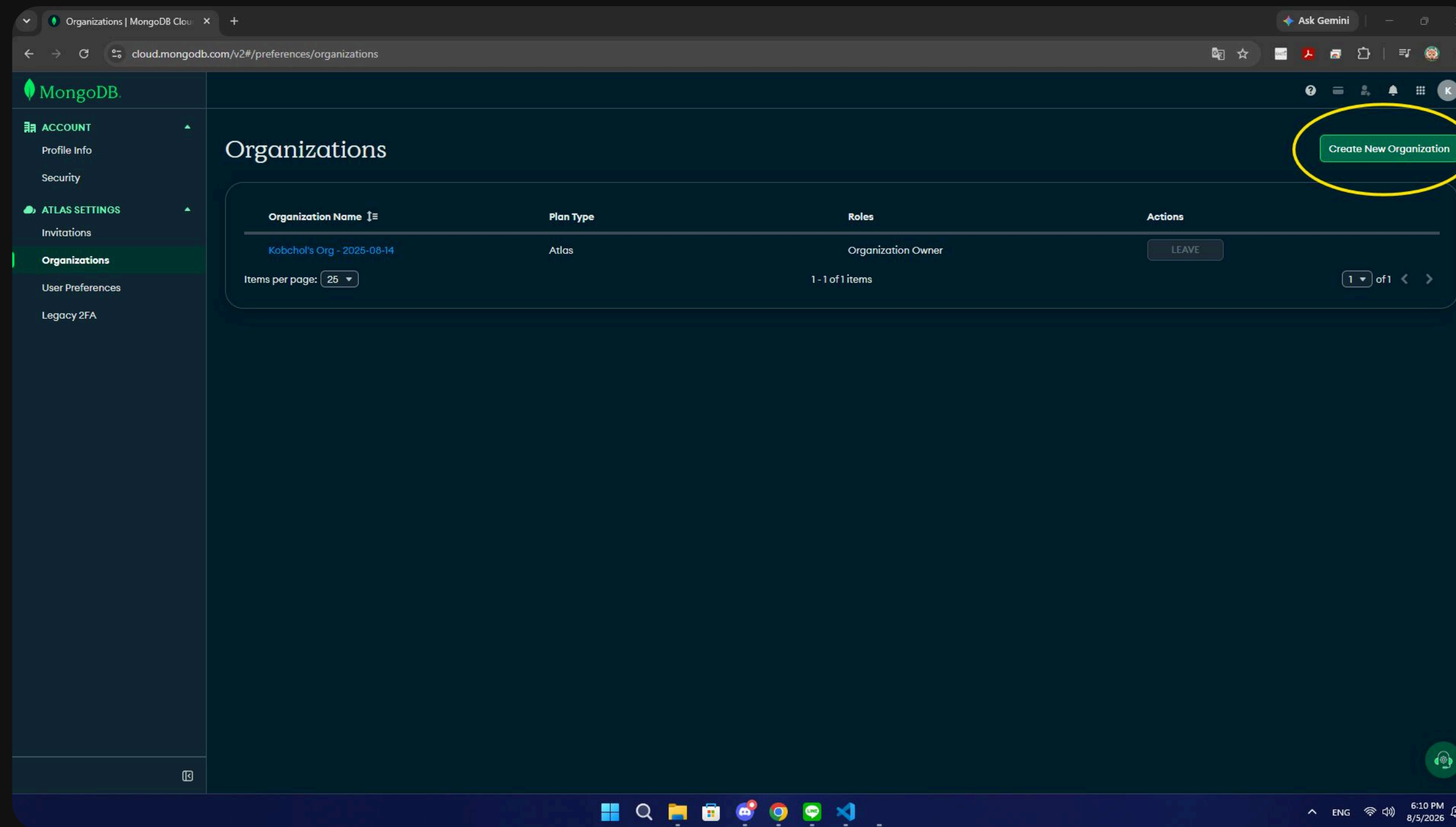
<https://goonlinetools.com/snapshot/code/#kpun1xza85maqmpgsw5nr>



สร้าง Database กัน



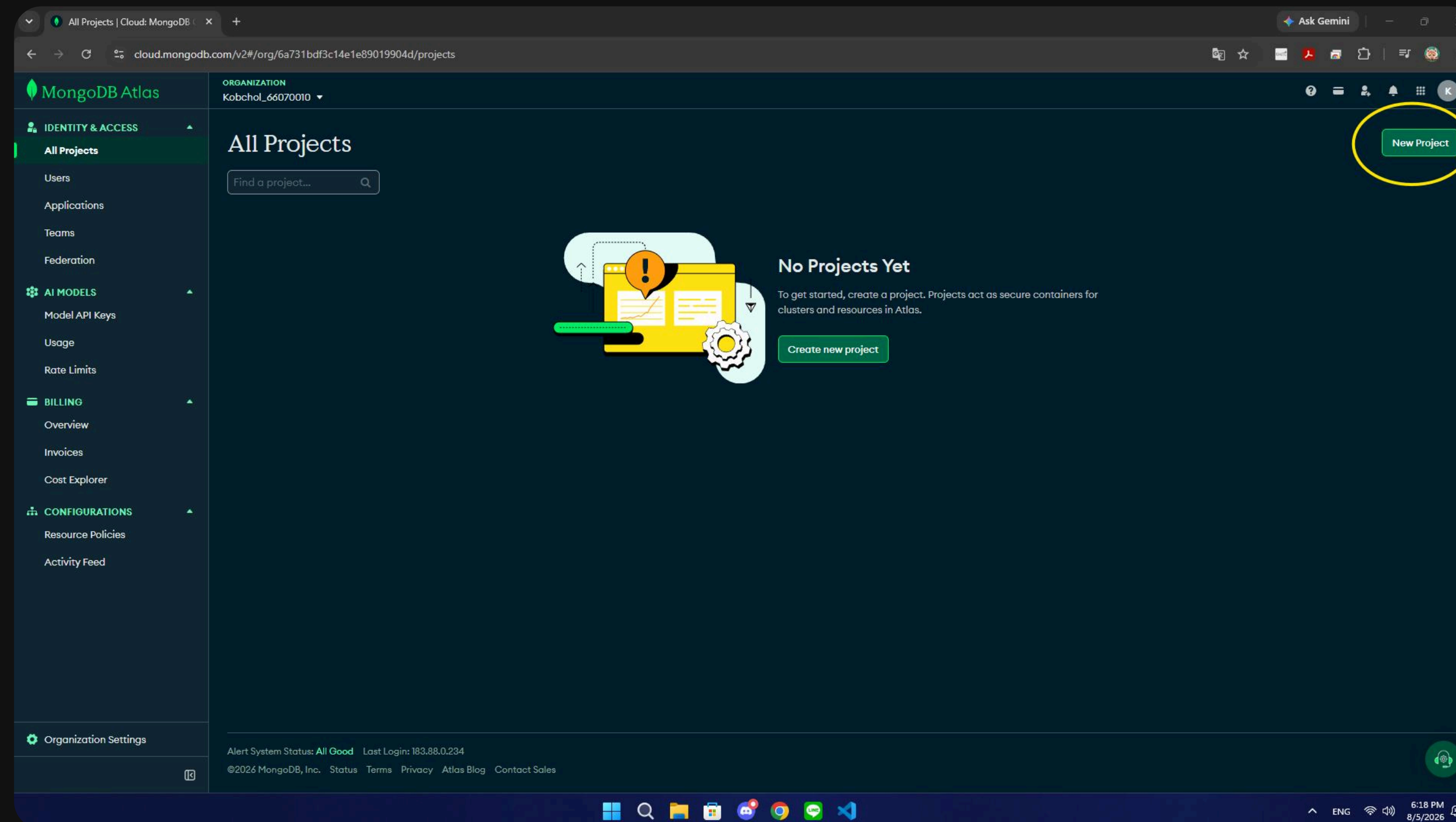
สร้าง Organization



<https://cloud.mongodb.com/v2#/preferences/organizations>

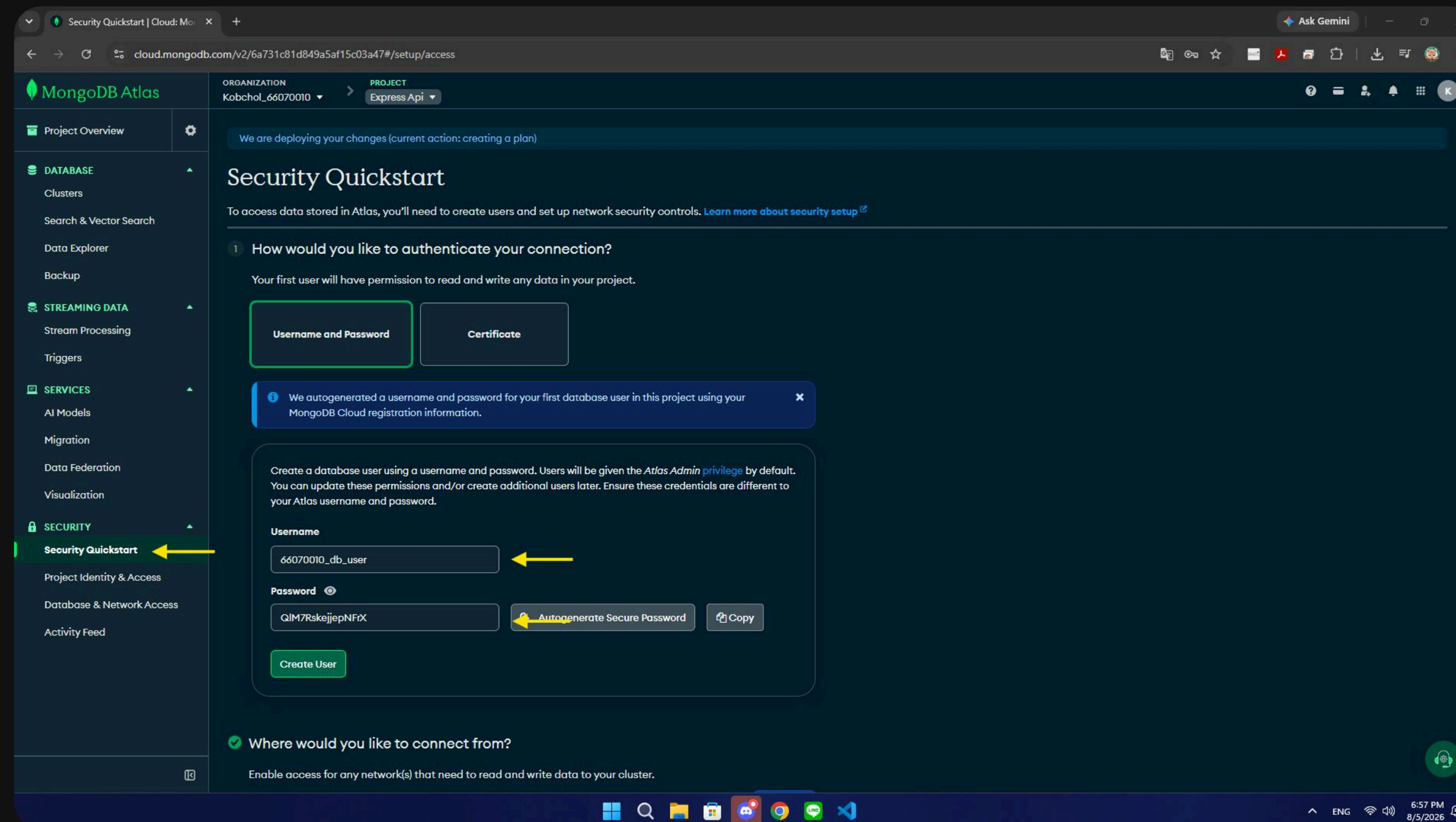


สร้าง Project ขึ้นมา



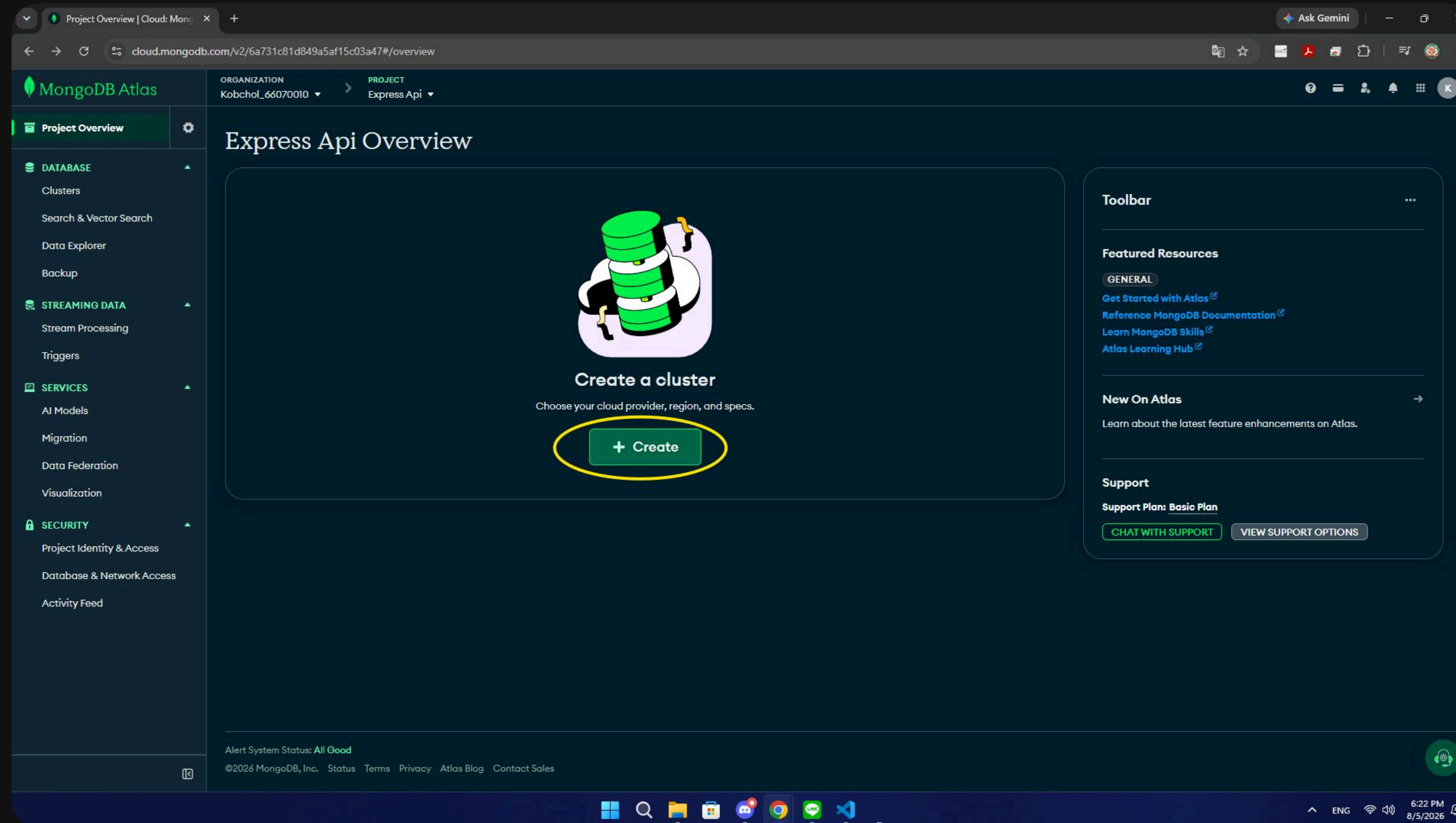


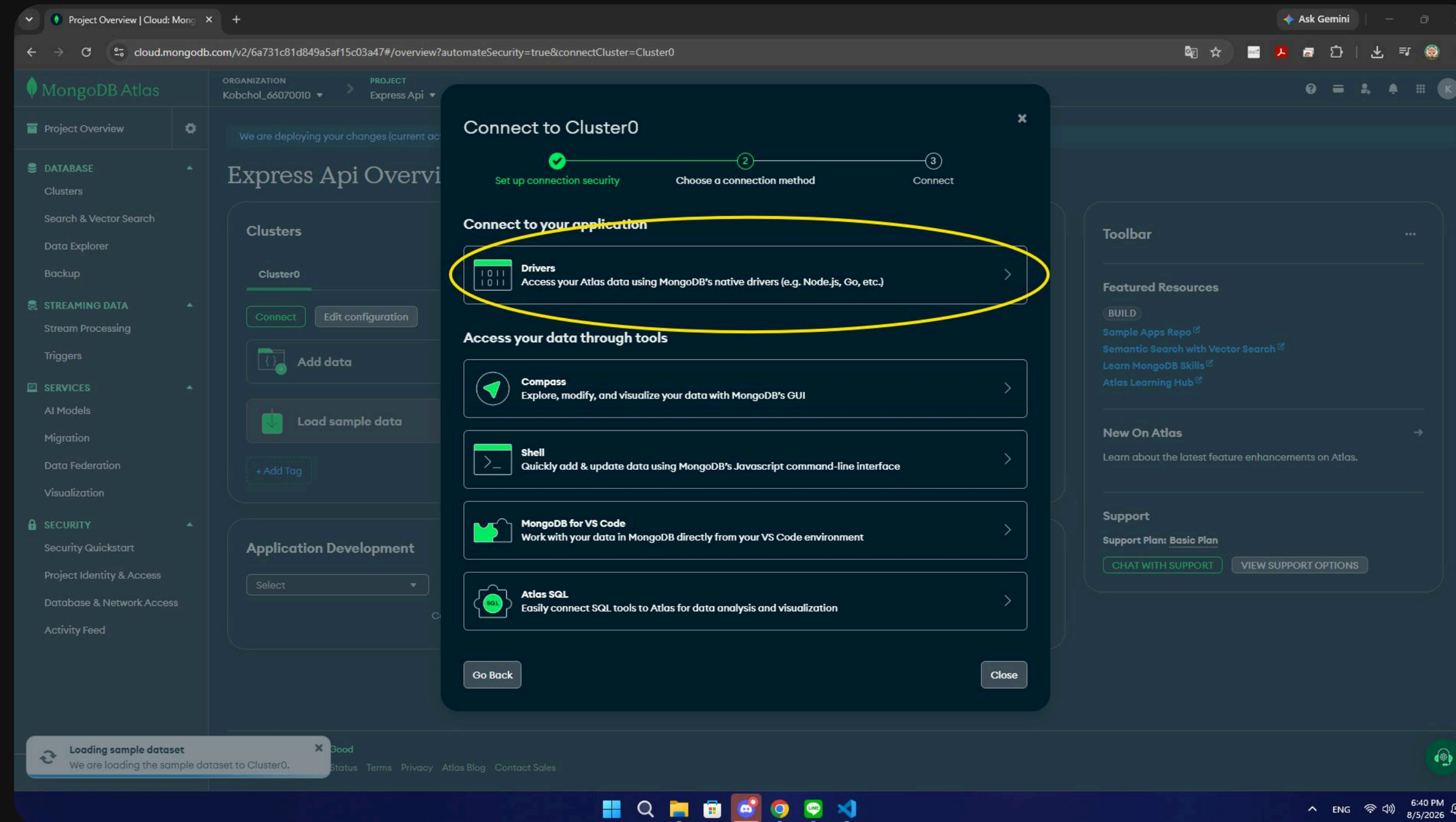
สร้าง User และ Password





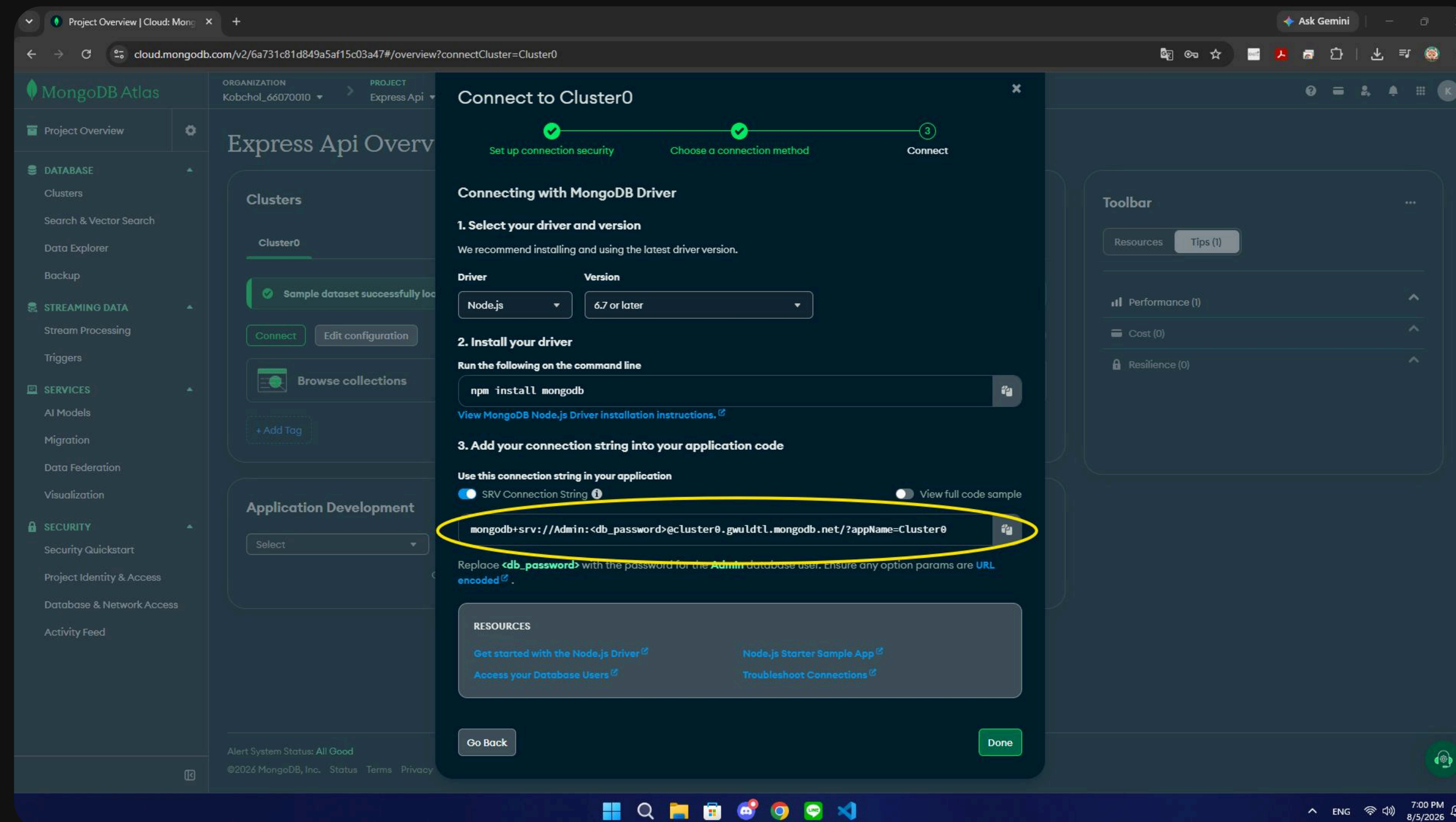
สร้าง Database







รับ URL เอาไปใส่ใน .env





ทดสอบดูว่าเชื่อมต่อไหม

```
Problems Output Debug Console Terminal Ports Azure Query Results

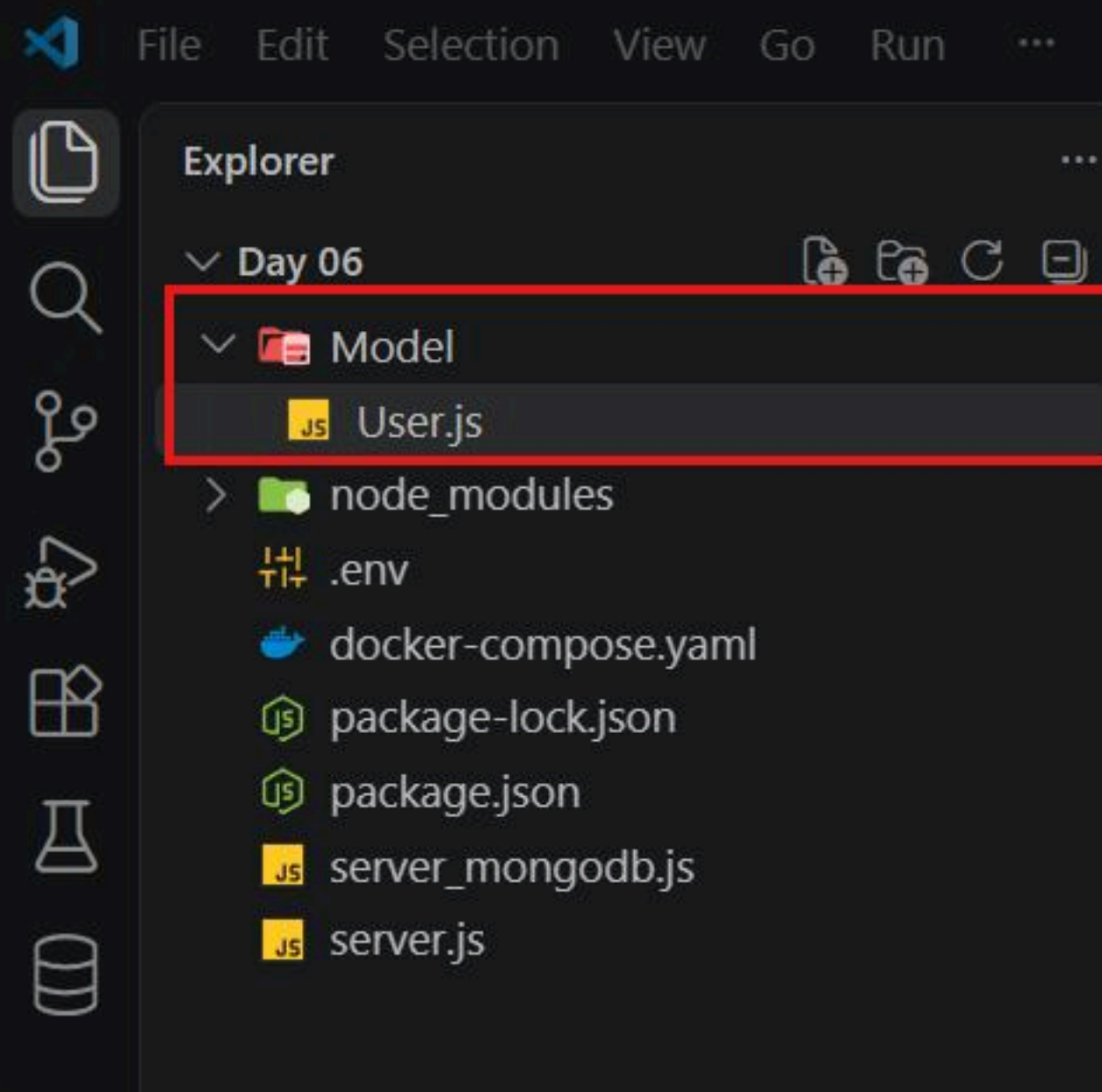
D:\Work In School\Term 4\TA\Day 06>npm run dev

> day-06@1.0.0 dev
> nodemon server_mongodb.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server_mongodb.js`
◇ injected env (7) from .env // tip: ◇ encrypted .env [www.dotenvx.com]
Server is running on http://localhost:3000
mongodb successfully connected!
Terminate batch job (Y/N)? Y
```



สร้าง Model ของ User





User.js

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema(
  {
    email: {
      type: String,
      required: true,
      unique: true,
      trim: true,
      lowercase: true,
    },
    fullname: {
      type: String,
      required: true,
    },
    password: {
      type: String,
      required: true,
    },
    deleted_at: {
      type: Date,
      default: null, // ถ้ายังไม่ลบจะเป็น null (สำหรับทำ Soft Delete)
    },
  },
  {
    timestamps: true, // จะสร้าง createdAt และ updatedAt ให้อัตโนมัติ
  }
);

module.exports = mongoose.model('User', userSchema);
```

<https://goonlinetools.com/snapshot/code/#hn1uptc9iilhyhdaqsth0s>



Import User Model ที่สร้างไว้ User.js

The screenshot shows the VS Code editor interface. On the left, the Explorer sidebar displays the project structure for 'Day 06', including a 'Model' folder with 'User.js', 'node_modules', '.env', 'docker-compose.yaml', 'package-lock.json', 'package.json', 'server_mongodb.js', and 'server.js'. The main editor area has several tabs open: 'server.js', 'server_mongodb.js', 'User.js', 'package.json', and '.env'. The 'server_mongodb.js' file is active, showing JavaScript code. Line 6, `const User = require('./Model/User');`, is highlighted with a red rectangle. The code includes imports for 'express' and 'mongoose', configuration for 'dotenv', and a MongoDB connection setup.

```
1  const express = require('express');
2  const mongoose = require('mongoose');
3
4  require('dotenv').config();
5
6  const User = require('./Model/User');
7
8  const app = express();
9  app.use(express.json());
10
11 // เชื่อมต่อ MongoDB
12 mongoose.connect(process.env.MONGODB_URI) // URL ของ MONGODB
13   .then(() => console.log('mongodb successfully connected!'))
14   .catch((err) => console.error(err));
15
```



มาสร้าง CRUD กัน



สร้าง Create

```
// --- ROUTES ---

// CREATE Route: เพิ่มผู้ใช้ใหม่
app.post('/create', async (req, res) => {
  const { email, name, password } = req.body;

  try {
    const newUser = await User.create({
      email,
      fullname: name,
      password,
    });

    return res.status(201).json({
      message: 'New user successfully created!',
      user: newUser,
    });
  } catch (err) {
    console.error('Error while inserting a user into the database', err);
    return res.status(400).json({ error: err.message });
  }
});
```

<https://goonlinetools.com/snapshot/code/#waj5cxhfyybx557rrhi1nb>



สร้าง Read กับ Read single users

```
// READ Route: ดึงเฉพาะผู้ใช้ที่ยังไม่ถูก Soft Delete
app.get('/read', async (req, res) => {
  try {
    // กรองเอาเฉพาะรายการที่ deleted_at เป็น null
    const users = await User.find({ deleted_at: null });
    return res.status(200).json(users);
  } catch (err) {
    console.error(err);
    return res.status(500).json({ error: 'Internal Server Error' });
  }
});

// READ Single User Route: ดึงผู้ใช้รายบุคคลด้วย email
app.get('/read/single/:email', async (req, res) => {
  const { email } = req.params;

  try {
    // หาผู้ใช้ตาม email และต้องยังไม่ถูกลบ
    const user = await User.findOne({ email, deleted_at: null });

    if (!user) {
      return res.status(404).json({ message: 'User not found or deleted' });
    }

    return res.status(200).json(user);
  } catch (err) {
    console.error(err);
    return res.status(500).json({ error: 'Internal Server Error' });
  }
});
```

<https://goonlinetools.com/snapshot/code/#cz2nwa8ijtjv0krkk02m>



สร้าง Update

```
// UPDATE Route: อัปเดตรหัสผ่าน (เฉพาะบัญชีที่ยังไม่ถูกลบ)
app.patch('/update/:email', async (req, res) => {
  const { email } = req.params;
  const { newPassword } = req.body;

  try {
    const updatedUser = await User.findOneAndUpdate(
      { email, deleted_at: null }, // เงื่อนไขการค้นหา
      { password: newPassword }, // ข้อมูลที่จะอัปเดต
      { new: true }               // คืนค่าข้อมูลใหม่หลังอัปเดต
    );

    if (!updatedUser) {
      return res.status(404).json({ message: 'User not found or deleted' });
    }

    return res.status(200).json({
      message: 'User password updated successfully!',
      user: updatedUser,
    });
  } catch (err) {
    console.error(err);
    return res.status(500).json({ error: 'Internal Server Error' });
  }
});
```

<https://goonlinetools.com/snapshot/code/#u6c5vki22jd5ucc1bbffs>



สร้าง Soft Delete

```

// SOFT DELETE Route: เปลี่ยนเป็นการใส่วันที่ปัจจุบันลงใน deleted_at
app.delete('/delete/:email', async (req, res) => {
  const { email } = req.params;

  try {
    const deletedUser = await User.findOneAndUpdate(
      { email, deleted_at: null }, // กรองเฉพาะบัญชีที่ยังใช้งานอยู่
      { deleted_at: new Date() }, // ใส่เวลาปัจจุบันแทน NOW() ของ SQL
      { new: true }
    );
    if (!deletedUser) {
      return res.status(404).json({ message: 'No active user found with that email!' });
    }

    return res.status(200).json({ message: 'User soft-deleted successfully!' });
  } catch (err) {
    console.error(err);
    return res.status(500).json({ error: 'Internal Server Error' });
  }
});
```

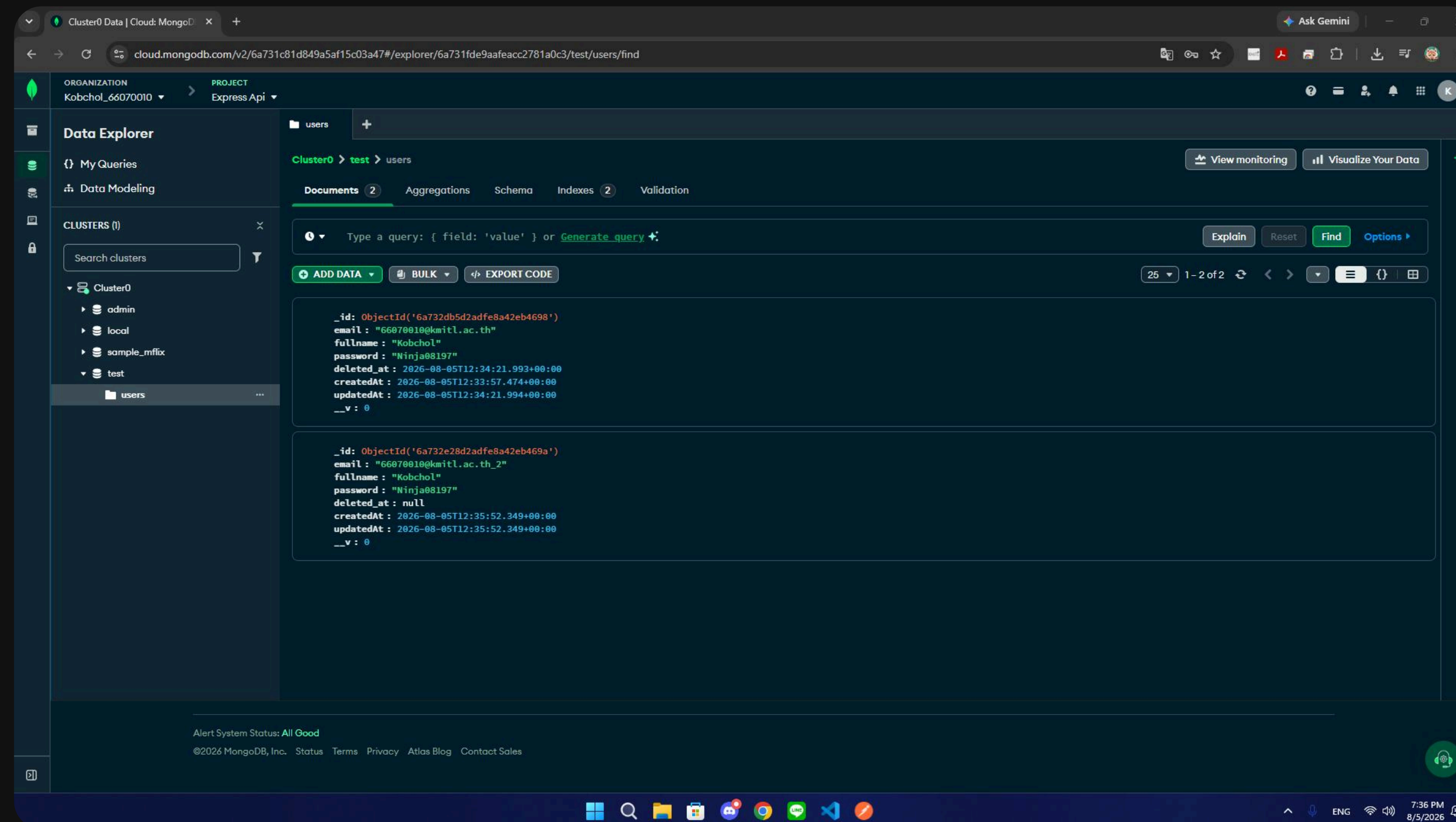
<https://goonlinetools.com/snapshot/code/#t82z1z16wgb9fzwtmfuuv>



ทดสอบใน Postman ดูว่าใช้งานได้ไหม



เช็คข้อมูลได้ใน Data Explorer





จบแล้วสำหรับ Week นี้ครับ